Do Language Models Know Which Constraints Belong Together?

Junbo Jacob Lian [1 2 3] **Kaixuan Chen** [3] **Hanzhang Qin** [1] **Huiling Chen** [4] **Chaoyu Zhang** [5] **Chung-Piaw Teo** [1]

Abstract

Dantzig–Wolfe reformulation strengthens the linear relaxation of a mixed-integer program by convexifying a chosen set of constraint families, and every candidate costs a column generation run. The choice is hard when families strengthen the bound only together, because a search that adds one family at a time then stops at the linear relaxation. The classical inequalities of lot sizing, network design and facility location rest on such zero-marginal groups. We prove that when each evaluation convexifies at most s of k families, any method that observes only bounds needs a covering number of evaluations, of order k^w, to find a hidden group of w families, also on actual Dantzig–Wolfe relaxations. A prediction that misses e members reduces the cost to $\Theta(k^e)$, through one staged search that does not need to know e, so the evaluations a prediction saves measure what the predictor knows. We study two sources of predictions: a one-pass lookahead at an optimal LP dual, which is provably blind to groups whose families share no variable, and a language model that reads the names and text of the model. JointDW verifies the latter with the staged search and interleaves it with the lookahead at a bounded loss. Zero-marginal groups appear in three of ten MIPLIB-NL models whose bound can be improved and in every one of 59 standard lot-sizing, network-design and facility-location instances, where GCG's detector convexifies a set with no gain on 38 of them. On MIPLIB-NL the strongest language models identify the groups about as often as the lookahead, their predictions rest on the names for textbook models, and reasoning about doubles the accuracy of the same model. Given every coefficient of a larger model, one reasoning model names the classical group in all but one answer, also when the names are permuted or removed, and needs fewer evaluations than the lookahead for pairs of families; a predictor trained on the same class matches this only after hundreds of labeled evaluations. On the benchmark instances the language models name the best pair more often than the lookahead and the detector, and the sets chosen by GPT-6.1 Sol come closest to the branch-and-price time of the best set. Code is available at github.com/junbolian/JointDW.

1. Introduction

Dantzig–Wolfe (DW) reformulation (Dantzig & Wolfe, 1960) replaces some constraints of a mixed-integer linear program (MILP) by the convex hull of their integer solutions, and column generation solves the resulting relaxation. Its bound is never weaker than that of the linear programming (LP) relaxation, and it is the basis of branch-and-price for lot sizing, vehicle routing and scheduling (Vanderbeck & Wolsey, 2010). The bound depends on which constraints are convexified, and the bound of a candidate is known only after a column generation run. Modelers make this choice from knowledge of the model class, for example by convexifying the balance and setup rows of a lot-sizing model together, while generic solvers such as GCG detect block structure in the matrix and reformulate automatically (Gamrath & Lübbecke, 2010; Bergner et al., 2015; Hojny et al., 2025). Without the modeler's knowledge a detector must compare candidates, and a model with k constraint families has 2^k of them. This paper asks how many evaluations that knowledge replaces, and whether a language model that reads the model can supply it.

The choice is hard for a specific reason, which Figure 1 shows on a lot-sizing instance with two items and two periods. Convexifying the balance rows, the setup rows or the capacity rows alone leaves the LP bound unchanged, while convexifying the balance and setup rows together closes most of the integrality gap. Together the two families imply the (l, S) inequalities of single-item lot sizing (Bárány et al., 1984), which neither implies alone. A search that adds one

[1]Institute of Operations Research and Analytics, National University of Singapore, Singapore [2]Wenzhou Buyi Pharmacy, Wenzhou, China [3]McCormick School of Engineering, Northwestern University, Evanston, IL 60208, USA [4]College of Computer Science and Artificial Intelligence, Wenzhou University, Wenzhou, China [5]Department of Decision Analytics and Operations, City University of Hong Kong, Hong Kong, China. Correspondence to: Hanzhang Qin <hzqin@nus.edu.sg>, Huiling Chen <chenhuiling.jlu@gmail.com>, Chung-Piaw Teo <bizteocp@nus.edu.sg>.

Preprint.

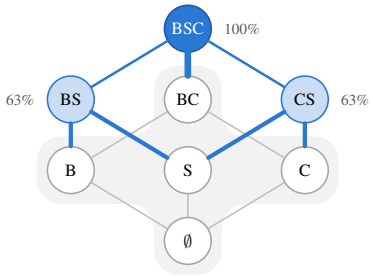


Figure 1. Share of the integrality gap closed by every family-level DW relaxation of a lot-sizing instance with two items and two periods, with B, S and C the balance, setup and capacity families. Edge width is proportional to the increase of the bound when one family is added, and gray edges add nothing. The shaded sets have gain zero, so a search that adds one family at a time while the bound improves never leaves $\emptyset$; the pairs BS and CS are zero-marginal groups.

family at a time while the bound improves therefore returns the LP relaxation, and a score that evaluates each family by itself sees no gain. We call a set of families with this property a zero-marginal group. The classical strong formulations of lot sizing, network design and facility location rest on such groups, since their inequalities need two families at once. What modelers know about these formulations is therefore the information that single-family evaluations cannot reveal. A general account of when families must be convexified together, and of what it costs to find them, has been missing.

Bound evaluations alone find such groups slowly. On binary programs whose DW relaxations we construct exactly, a bound reveals only whether the convexified set contains a hidden group, and every method needs a covering number of evaluations to find it. A prediction of the group removes a factor of order k from this cost for every member it gets right. With $k = 20$ families, evaluations of at most six families and a hidden group of three, a greedy covering needs 85 evaluations in the worst case, and a correct prediction needs one (Table 4). The evaluations a prediction saves thus measure what the predictor knows in units of computation, not by agreement with a reference answer. Two sources of predictions exist before any column generation run. Solving each block once at an optimal LP dual gives a lower bound on its gain, and we prove that this lookahead cannot see groups whose families share no variable. The text of the model is the other source: a modeler who knows lot sizing knows that balance and setup rows belong together, and language models now read such text when they formulate models (Huang et al., 2025; AhmadiTeshnizi et al., 2024; Liang et al., 2026; Lian et al., 2026b), check reformulations (Zhai et al., 2025; Robbins et al., 2026) and configure solvers (Lawless et al., 2024; Nie et al., 2026; Xu et al., 2026). Whether they know which constraints belong together is open; a correct prediction may come from the meaning of a model or only from familiar names, and its value depends on the search that checks it. Our contributions are the following.

- **Zero-marginal groups.** We define the family supports of a valid inequality and the zero-marginal groups of constraint families, and we distinguish groups that need an inequality with a joint support from groups that arise from the geometry of the LP (Section 3).
- **The cost of finding them.** When an evaluation may convexify at most s of k families, every method that observes only bounds needs $\Theta(k^w)$ evaluations to find a group of w families, also on DW relaxations of stable-set programs. Randomization saves at most a factor of two, and greedy selection never finds the group (Theorem 1). A prediction that misses e members reduces the cost to $\Theta(k^e)$ through one staged search that does not need to know e, and the search costs only a lower-order term more when the prediction is entirely wrong (Theorem 2).
- **Two sources of predictions.** The one-pass lookahead at an optimal LP dual is provably blind to groups whose families share no variable (Proposition 1). JointDW takes the prediction of a language model, verifies it with the staged search, and interleaves it with the lookahead at no more than twice the cost of the better of the two (Section 5).
- **A controlled measurement.** We report a census of zero-marginal groups in MIPLIB-NL and on 59 standard benchmark instances, and we measure five language-model configurations on instances with exact gains against the lookahead, GCG's detector and a predictor trained on labeled evaluations. Controlled presentations separate knowledge of a model from familiarity with its names, and we vary reasoning and whether the coefficients are shown (Section 6). Code and data are released.

The language models know what the textbooks say about these groups, and on textbook models they know it through the names: once the family names are permuted among the rows, GPT-6.1 Sol names no true group on the templates. Zero-marginal groups are common in MIPLIB-NL, greedy selection misses the best decomposition on most models that have one, and on standard benchmark instances GCG's detector convexifies the block structure of the matrix rather than the group that carries the gain. On MIPLIB-NL the strongest language models identify the groups about as often as the lookahead and on partly different models, and reasoning roughly doubles the accuracy of the same model. Given every coefficient of a larger model, one reasoning model names the classical group of each class, also with the names permuted or removed, and needs fewer evaluations than the lookahead for pairs of families. A predictor trained on the gain tables of the same class matches this only after hundreds of labeled evaluations. Because the interleaved order bounds the loss from a wrong prediction, a more accurate

language model can be adopted as soon as one exists.

2. Related Work

Choosing DW reformulations. Geoffrion's theorem (Geoffrion, 1974) identifies the bound of a Lagrangian relaxation, and hence of a DW reformulation, with the optimum over the convex hull of the kept constraints intersected with the dualized constraints. Guignard & Kim (1987) showed that intersecting two such hulls can dominate both relaxations. Bastubbe et al. (2018) computed the bounds of all constraint subsets of small integer programs and found few distinct bounds, and Lübbecke & Witt (2018) and Witt (2020) studied which valid inequalities DW reformulations imply. GCG detects decompositions from the structure of the constraint matrix (Gamrath & Lübbecke, 2010; Bergner et al., 2015); one of its detectors enumerates subsets of constraint classes, which are the families of this paper (GCG developers, 2026a), and its strong score solves each block once at a mixture of LP duals and random duals (GCG developers, 2026b). Learned detectors decide from structural features whether and how to decompose (Kruber et al., 2017; Basso et al., 2020; Basso & Ceselli, 2023); they are trained on instances of one distribution and see only the constraint matrix, while a language model needs no training instances and also reads the names and the description. We train a predictor of this kind on our own gain tables and compare the two (Section 6). Single-family evaluations give no signal on a zero-marginal group, and Proposition 1 states which groups a one-pass evaluation at an optimal LP dual cannot see.

Language models for optimization. Language models formulate optimization models from text (Huang et al., 2025; AhmadiTeshnizi et al., 2024; Liang et al., 2026). Generated formulations are checked by perturbation responses (Lian et al., 2026b), agreement across model families (Lian et al., 2026a), solver feedback (Zhu et al., 2026), and variable mappings, Lean proofs or SMT solvers (Zhai et al., 2025; Robbins et al., 2026; Bhattacharyya & Baranwal, 2026). MIPLIB-NL pairs MIPLIB 2017 instances with natural-language specifications (Li et al., 2026). Closer to our question, language models evolve acceleration inequalities and faster formulations (Yazdani et al., 2026; Yuan et al., 2026) and cut selection and branching rules (Nie et al., 2026), configure cutting-plane separators (Lawless et al., 2024) and write constraint handlers that recover global constraints in MIPLIB models (Xu et al., 2026). FrontierOR (Kong et al., 2026) and Baek (2026) evaluate algorithm design for OR problems, DecompGym asks coding agents to implement a prescribed decomposition (Cheng et al., 2026), and Le et al. (2026) learn the multipliers of a fixed Lagrangian relaxation. In these works the decomposition is absent or given; we study how to choose it and measure the contribution of a language model by its number of misses, which Theorem 2 converts into search cost.

Search with predictions. Algorithms with predictions are analyzed by their consistency and robustness (Mitzenmacher & Vassilvitskii, 2022); learning-augmented query policies trade one against the other (Erlebach et al., 2020), and the cost of graph search with predictions grows with the number of wrong predictions (Banerjee et al., 2022). Our oracle reveals whether a query contains a hidden group, as in learning a hidden hypergraph with edge-detecting queries (Angluin & Chen, 2006) and in group testing with tests of bounded size (Gandikota et al., 2019), and the tractability budget makes covering designs (Fort & Hedlund, 1958; Schönheim, 1964; Rödl, 1985; Gordon et al., 1995) the exact measure of cost.

3. Family-Level DW Relaxations

Consider $\min\{cx : x \in X\}$ with

$$X = \{x \in \mathbb{Z}^p \times \mathbb{R}^q : A_f x \geq b_f \text{ for } f \in F,\ \ell \leq x \leq u\},$$

rational data, finite bounds that are integral for the integer variables, and $X \neq \emptyset$. The rows are partitioned into $k = |F|$ constraint families; in a modeling language, a family is one indexed constraint such as all balance rows. For $T \subseteq F$ let X_T keep only the rows of the families in T, let LP be the LP relaxation of X, and let

$$P_T = \mathrm{LP} \cap \mathrm{conv}(X_T), \qquad z(T) = \min_{x \in P_T} cx. \quad (1)$$

The gain of T is $g(T) = z(T) - z(\emptyset)$. By Geoffrion's theorem, $z(T)$ is the bound of the DW reformulation that convexifies the rows of T and keeps those of $F \setminus T$ in the master problem. Each connected component of the rows of T forms one block. Since $X_{T'} \subseteq X_T$ when $T \subseteq T'$, the gain g is nondecreasing, $g(\emptyset) = 0$, and $g(F) = z_{\mathrm{IP}} - z_{\mathrm{LP}}$. Convexifying F solves the MILP itself, and convexifying many families makes the pricing problems hard. We model tractability by a family $\mathcal{T}$ of sets of families, closed under taking subsets, with $\mathcal{T}_s = \{T : |T| \leq s\}$ as the main case.

Definition 1 (family support)**.** Let $\alpha x \geq \beta$ be valid for $\mathrm{conv}(X)$. The sets T for which $\alpha x \geq \beta$ is valid for P_T form an up-set that contains F. Its minimal elements are the family supports of the inequality, and a family support with at least two families is a joint support.

The polytope P_T is the intersection of LP with all valid inequalities that have a family support inside T (Proposition 2 in Appendix A). Hence $g(T) > 0$ exactly when these inequalities together cut off the whole LP-optimal face.

Definition 2 (zero-marginal group)**.** Let $B \subseteq F$ and let $W \subseteq F \setminus B$ be nonempty. W is a zero-marginal group at

B if $g(B \cup W) > g(B)$ and $g(B \cup S) = g(B)$ for every $S \subsetneq W$; its order is $w = |W|$. For $w \geq 2$ let $Q_W = \bigcap_{S \subsetneq W} P_{B \cup S}$. The group is LP-geometric if $\min\{cx : x \in Q_W\} > z(B)$ and polyhedral otherwise.

The two kinds differ in what produces the gain. In a polyhedral group the gain needs an inequality that is not valid for Q_W. Every family support of such an inequality inside $B \cup W$ contains all of W, because a support that misses some $f \in W$ lies in $B \cup (W \setminus \{f\})$, so at $B = \emptyset$, W is a joint support of an inequality that cuts off an LP optimum. In an LP-geometric group, inequalities whose supports each miss a member of W cut off every LP optimum only together, for example when two bound rows each fix one variable and the LP shifts its weight to the other (Example 1 in Appendix B). Both groups in Figure 1 are polyhedral.

Classical inequalities as joint supports. In lot sizing, the balance and setup rows of an item together imply the (l, S) inequalities (Bárány et al., 1984). In network design, the flow-balance rows summed over one side of a cut bound the flow across the cut from below, and with the arc-capacity rows and the integrality of the opening variables this gives the cut-set inequalities (Atamtürk, 2002; Raack et al., 2011). In capacitated facility location, the capacity rows summed with the assignment rows give a knapsack constraint on the opening variables, whose cover inequalities need both families (Aardal et al., 1995). A modeler who knows the class can therefore name a candidate group without evaluating a bound, and Section 6 tests whether a language model can.

4. The Cost of Choosing a Decomposition

We analyze search in an oracle model. A method observes the program only through g: it queries sets $T \in \mathcal{T}$ one at a time, each chosen after the previous answers, and observes $g(T)$. Its cost is the number of queries until it has queried a set of maximal gain over $\mathcal{T}$; it need not certify optimality, so the lower bounds below hold for the easier task. For a set W of w families let $g_W(T) = \frac{1}{2}[W \subseteq T]$, and let $N(\mathcal{T}, w)$ be the least number of members of $\mathcal{T}$ that together cover every w-set contained in some member of $\mathcal{T}$. For $\mathcal{T}_s$ this is the covering number $\mathrm{C}(k, s, w)$, the least number of s-subsets of a k-set covering all its w-subsets.

Theorem 1 (covering). *Let $2 \leq w \leq s < k$.*

(a) For every $W \subseteq F$ with $|W| = w$ there is a binary program with k constraint families whose gain function is g_W. It is the edge formulation of stable set, with objective $\min -\sum_v x_v$, on an odd cycle of length at least w and on $k - w$ further paths, all on disjoint sets of vertices. The edges of the cycle are split into w paths, which form the families of W.

(b) Every deterministic method needs $N(\mathcal{T}, w)$ queries in the worst case over $\{g_W\}$, and querying a minimum covering attains this. For $\mathcal{T} = \mathcal{T}_s$, every randomized method needs more than $\frac{1}{2}\binom{k}{w}/\binom{s}{w}$ queries in expectation for some W.

(c) $\binom{k}{w}/\binom{s}{w} \leq \mathrm{C}(k, s, w) \leq \left(1 + \ln \binom{s}{w}\right)\binom{k}{w}/\binom{s}{w}$, and $\mathrm{C}(k, s, w) = (1 + o(1))\binom{k}{w}/\binom{s}{w}$ for fixed s and w as $k \to \infty$. A method that does not know w can query minimum coverings for $w' = 1, 2, \ldots$ in turn and pays $\sum_{w' \leq w} \mathrm{C}(k, s, w') = (1 + o(1))\mathrm{C}(k, s, w)$.

(d) Adding one family at a time while some addition has a positive marginal gain stops at $\emptyset$ on every g_W.

Part (a) holds because a forest is bipartite: convexifying the rows of T changes nothing unless T contains every family of W, and these families together form the odd cycle, which then contributes the difference of $\frac{1}{2}$ between its LP value and the optimum over its stable-set polytope. On these programs a bound carries one bit, whether the query contains W, so decomposition search under a tractability budget is the search for one hidden hyperedge with edge-detecting queries of bounded size; without the budget, such a hyperedge is found with $O(w \log k)$ queries (Angluin & Chen, 2006). Part (b) is an adversary argument, and part (c) follows from counting, from the greedy covering bound of Lovász (1975) and Stein (1974), and from Rödl's theorem (Rödl, 1985); Appendix A gives the proofs. The group is visible in the constraint matrix of these programs, so the theorem concerns the information in bound evaluations, and Theorem 2 measures what reading the formulation is worth.

Theorem 2 (value of a predicted group). *Let $2 \leq w \leq s < k$, let $g = g_W$ be a gain function of Theorem 1 with hidden group W, and let a prediction $P \subseteq F$ with $|P| = w$ miss $e = |W \setminus P|$ members of W. For $0 \leq e' \leq w$ let $A_{e'} = \{a \in \mathbb{Z} : w - e' \leq a \leq \min(w, s - e')\}$ and $f_{e'}(a) = \binom{a}{w-e'}\binom{s-a}{e'}$. Let $\mathrm{C}(n, a, t)$ be the least number of sets of at most a elements of an n-set that cover its t-subsets; it equals 1 when $t = 0$ or $t \leq n \leq a$, and a stage with $e' > k - w$ is empty.*

(a) Staged search. *Stage e' queries every $A \cup R$, where A is in a minimum covering of the $(w - e')$-subsets of P by a-sets and R is in a minimum covering of the e'-subsets of $F \setminus P$ by sets of at most $s - a$ elements, with $R = \emptyset$ for $e' = 0$. The split $a = a_{e'} \in A_{e'}$ gives the smallest stage, and the stages run for $e' = 0, 1, \ldots, w$ in turn. Without knowing e, this search finds W within $\sum_{e'=0}^{e} \mathrm{C}(w, a_{e'}, w - e')\,\mathrm{C}(k - w, s - a_{e'}, e')$ queries.*

(b) Lower bound. *Every deterministic method, even one that knows e, needs at least L_e queries in the worst case, where $L_e = \binom{w}{e}\binom{k-w}{e}/\max_{a \in A_e} f_e(a)$. If $e(s+1) \leq w(s - w + 1)$, then $L_e = \binom{k-w}{e}/\binom{s-w}{e}$, and $L_w = \binom{k-w}{w}/\binom{s}{w}$.*

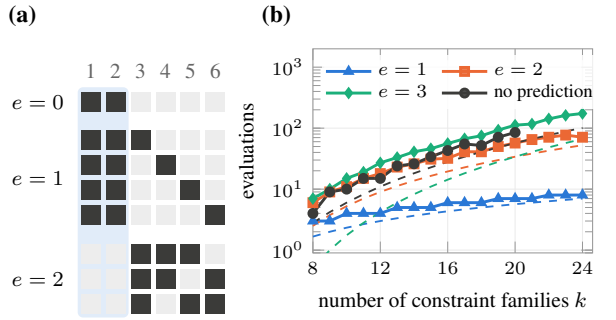


Figure 2. (a) Queries of the staged search of Theorem 2(a), from top to bottom, for $k = 6$, $s = 3$, $w = 2$ and the prediction $P = \{1, 2\}$ (shaded columns): one query per row, with its families in dark cells. The queries of stage e contain every hidden group with e members outside P, so a correct prediction is confirmed by the first query. (b) Evaluations to find a hidden group of $w = 3$ families with $s = 6$. Solid lines: for predictions that miss e members, the queries in the stages of the staged search built from greedy coverings; without a prediction, the worst case of a greedy covering. Dashed lines of the same color: the lower bounds L_e of Theorem 2(b) and $\binom{k}{3}/\binom{6}{3}$ of Theorem 1 (values in Table 4).

(c) No price for robustness. *For fixed s and w and $k \to \infty$, the cost in (a) is $(1 + o(1))\rho_e L_e = \Theta(k^e)$, where $\rho_e \geq 1$ depends only on (s, w, e) and equals 1 for $e = 0$, for $e = w$, and whenever $e(s + 1) \leq w(s - w + 1)$. For $e = w$ the cost is at most $\mathrm{C}(k, s, w) + O(k^{w-1})$.*

Every hidden group with e members outside P lies in some query of stage e (Figure 2(a)). Part (b) counts the hidden groups that a promise of e misses leaves open and bounds how many of them one query can contain, and part (c) compares the last stage of (a) with (b) through Rödl's asymptotics. Each correctly predicted member thus divides the cost by a factor of order k (Figure 2). In learning-augmented algorithms, consistency usually costs robustness (Mitzenmacher & Vassilvitskii, 2022; Erlebach et al., 2020); here the same search costs one query when the prediction is right, and only a lower-order term more than prediction-free search when it is entirely wrong. For finite k the earlier stages add an overhead, for example 56 against 43 queries at $k = 16$ and $e = 3$ (Table 4), and interleaving the staged order with a covering order caps the cost at twice the smaller of the two.

Side information from LP duals. Greedy selection over $\mathcal{T}_s$ reaches at least a fraction $1 - \exp(-\gamma)$ of the best gain when the submodularity ratio γ of g is positive (Das & Kempe, 2011), and a zero-marginal group at $\emptyset$ of order $2 \leq w \leq s$ forces $\gamma = 0$. A cheaper signal than column generation solves each block of T once at an optimal dual y of the LP relaxation, and we write $\ell_y(T)$ for the resulting increase of the bound.

Proposition 1 (one-pass lookahead)**.** *For every optimal dual y of the LP relaxation, $0 \leq \ell_y(T) \leq g(T)$, the function ℓ_y is nondecreasing, and $\ell_y(T_1 \cup T_2) = \ell_y(T_1) + \ell_y(T_2)$ whenever no variable appears in a row of T_1 and in a row of T_2. Consequently ℓ_y vanishes on every zero-marginal group at $\emptyset$ of order at least two whose families pairwise share no variable, every such group is LP-geometric, and every superset of a set receives a value at least as large as the set.*

A polyhedral group therefore has two families that share a variable, and on such a group the value of ℓ_y depends on the dual. On the instance of Figure 1, ℓ_y of capacity with setup is zero at the dual returned by HiGHS and stays below the gain at every optimal dual (Appendix B). The order of decreasing ℓ_y is the prediction of the LP itself, and GCG's strong score uses a yes-or-no version of it (GCG developers, 2026b); taking for each set the largest value over several optimal duals gives a stronger order, and Section 6 evaluates both. Lemma 3 in Appendix A proves the proposition.

Algorithm 1 JointDW

Require: MILP with constraint families F and its presentation; budget s; time limit τ per evaluation; N evaluations
1: $\mathcal{A} \leftarrow$ answer of the language model to the presentation of the MILP, with no bound shown
2: $P \leftarrow$ first joint support in $\mathcal{A}$, or the families that its top-ranked reformulation convexifies if $\mathcal{A}$ lists none
3: $O_1 \leftarrow$ staged order of Theorem 2(a) around P
4: $O_2 \leftarrow$ covering orders for $w' = 1, \ldots, s$ in turn
5: $O \leftarrow$ alternate the entries of O_1 and O_2, skipping repeats
6: **for** the first N sets T in O **do**
7: run column generation at the root node on the DW reformulation that convexifies T with time limit τ, and record its bound, a valid lower bound even if it has not converged
8: **end for**
9: **return** the set T with the best recorded bound

5. JointDW

Proposition 1 leaves groups whose families share no variable, and polyhedral groups on which the dual gives no signal, to information that does not come from the LP. JointDW takes it from a language model that reads the text of the model: it asks which families must be convexified together and checks the answer with the staged search of Theorem 2 (Algorithm 1). The prediction replaces what bound evaluations cannot do efficiently, and the search limits the cost of a wrong prediction. As in solver-based checks of generated formulations (Lian et al., 2026b;a), the answer enters the result only through computation: a predicted group counts only after bound evaluations confirm its gain.

Prediction and verification. The language model receives the description and the formulation of the model when they exist, and tables of the constraint families and

variable families with their sizes, senses, constraint types and variable types, but no bound, optimum or solver output (Appendix D). Its answer is one JSON object: for each family, whether convexifying it alone strengthens the bound and whether it has the integrality property; the joint supports, most important first, each with the class of inequalities it generates; and up to three ranked DW reformulations, each given by the families kept in the master. The anchor P is the first joint support, the predicted group of Theorem 2, or the families that the top-ranked reformulation convexifies if the answer lists none. An evaluation that does not finish within τ keeps its bound as a valid lower bound. On the gain functions of Theorem 1, Theorem 2 and the interleaving bound the cost by twice the smaller of the staged cost, $O(k^e)$ for a prediction with e misses, and the cost of prediction-free search. The order of Algorithm 1 can also be alternated with the lookahead order of Proposition 1, skipping repeats; we call the result the interleaved order, and by the interleaving argument of Appendix A it evaluates a best set within twice the smaller cost on every instance.

What the prediction is read from. A correct prediction can come from knowledge of the model class, from the names alone, or from cues in the statistics. Four presentations of the same model separate these sources: the *original* presentation shows everything, *names only* withholds the description and the formulation, *permuted* also permutes the family names among the families while the statistics stay with their rows, and *anonymized* replaces the names by $F_1, \ldots, F_k$ and $V_1, \ldots, V_m$. Answers are mapped back to the true families before scoring, so predictions that follow the names move to the wrong families under the permutation, and predictions that follow the statistics survive both changes.

6. Experiments

Instances. The fifteen templates of Appendix C have exact gains for every set of families, computed from explicit LPs whose optimal bases are verified in rational arithmetic (Appendix B): nine lot-sizing, network-design and facility-location instances whose groups are the joint supports named in Section 3, two generalized assignment instances without a joint support, which measure false alarms, and four hidden-group programs from Theorem 1(a), whose presentation carries no information about the group. MIPLIB-NL pairs MIPLIB 2017 instances with natural-language specifications and reconstruction scripts (Li et al., 2026), and we keep the 108 exported models with two to six constraint families. GCG on SCIP 10 (Hojny et al., 2025) evaluates every decomposition with at most $k-1$ families at the root node, under a time limit that acts as the tractability budget (Appendix E). A zero-marginal group is certain if every zero gain below it comes from a converged evaluation and possible otherwise, and the 102 models with a bounded evaluation are scored. The 24 rich models, eight per class, are connected programs of lot sizing, network design and facility location with seven to nine interacting families. They place the classical group among further modeling features, so that a covering of all pairs is large, and their gains are exact for every set of at most three families (Appendix C). The 59 benchmark instances are the standard formulations of the same three classes on published data: the 20 smallest capacitated lot-sizing instances of Trigeiro et al. (1989), 20 fixed-charge network-design instances of the R set of Crainic et al. (2001), and the 19 capacitated facility-location instances of the OR-Library (Beasley, 1990) with a positive LP gap, each with the three families of its class. GCG computes their gains for sets of at most two families as for MIPLIB-NL, and full branch-and-price with GCG, limited to 1800 seconds, measures what the choice is worth in solve time (Appendices C and E). The templates and MIPLIB-NL measure whether a prediction is right, the rich models what it saves when a covering of all pairs is large, and the benchmark instances what a choice costs in seconds.

Language models. Five configurations from four providers answer every instance and presentation three times: Claude Sonnet 5.5 with its default thinking, GPT-6.1 Sol with medium reasoning effort, Grok 4.20 (Reasoning), Grok 4.20 (Non-Reasoning), and DeepSeek-V4-Pro in non-thinking mode. All calls use OpenAI-compatible chat endpoints with pinned model identifiers, and every response is cached (Appendix D). Two prompt variants are asked in the original presentation only: the ranked variant also asks for up to five groups, most likely first, each with a probability, and the data variant adds the complete instance with all coefficients, on the rich models and the benchmark instances.

Measures. On an instance with a true joint support, hit@1 records whether the first predicted joint support is true and recall is the share of true joint supports that the answer lists; rates average the samples of an instance and then the instances. The lookahead order ranks the sets of at most $k-1$ families by decreasing ℓ_y at the dual returned by HiGHS and is scored as an answer with one joint support. The five-dual lookahead order takes for each set the largest value over this dual and four further optimal duals (Appendix E). Search cost is the number of evaluations until a set of maximal gain among the sets of at most s families is evaluated. It is averaged over random orders of the families. The baselines are a random order, greedy selection, the covering order, which queries greedy coverings of the pairs, triples and larger sets in turn, the two lookahead orders, the staged search anchored at GCG's own decomposition, and a learned predictor: gradient-boosted trees in the manner of Basso & Ceselli (2023) that score a set from structural features

Table 1. Prediction of joint supports in the original presentation. Templates: the nine lot-sizing, network-design and facility-location instances. MIPLIB-NL: the 17 scored models with a certain joint group. False alarms: the two assignment instances. Standard errors, the other presentations and the five-dual lookahead order: Appendix F.

	Templates		MIPLIB-NL		False
Configuration	hit@1	recall	hit@1	recall	alarms
Claude Sonnet 5.5	0.70	0.74	0.57	0.56	0.17
GPT-6.1 Sol	1.00	0.96	0.53	0.53	0.00
Grok 4.20 (Reasoning)	0.67	0.67	0.55	0.55	0.00
Grok 4.20 (Non-Reasoning)	0.33	0.17	0.29	0.29	0.00
DeepSeek-V4-Pro (Non-Thinking)	0.30	0.26	0.27	0.37	0.33
Lookahead order	0.67	0.44	0.53	0.44	–

Table 2. Mean number of evaluations until a best decomposition is evaluated when each evaluation may convexify at most s families, on the instances with a true joint support. The lowest value of each column is in bold. Greedy selection stops at the LP relaxation on every template and on most MIPLIB-NL models and is not shown. Standard errors, the five-dual lookahead order and the interleaved orders: Appendix F.

	Templates			MIPLIB-NL		
Order	$s=2$	3	$k-1$	2	3	$k-1$
Random, expected	4.57	4.56	3.82	4.82	5.70	5.96
Covering	2.86	2.33	1.82	3.08	2.84	2.11
Lookahead	**2.00**	1.92	1.92	2.18	2.12	2.29
JointDW with						
Claude Sonnet 5.5	2.47	2.11	1.71	2.20	2.26	1.91
GPT-6.1 Sol	2.07	**1.72**	**1.16**	**1.79**	**1.99**	**1.78**
Grok 4.20 (Reasoning)	2.48	2.10	1.59	2.42	2.32	1.90
Grok 4.20 (Non-Reasoning)	2.92	2.68	2.16	2.76	2.75	2.04
DeepSeek-V4-Pro (Non-Thinking)	3.14	2.69	2.14	2.51	2.51	2.08

of its decomposition, with or without its lookahead value, trained on the gain tables of other instances (Appendix F). On the benchmark instances each method contributes one set, and we record whether it is the true best pair and the time GCG needs to solve the instance by branch-and-price on it. Standard errors come from bootstrap resamples of the instances, with instances of identical prompts drawn together, and a difference exceeds sampling variation when its 95 percent bootstrap interval excludes zero (Appendix F).

Groups that gain only jointly are common. Three of ten MIPLIB-NL models whose bound can be improved have a certain zero-marginal group at $\emptyset$, and another third have a possible one (Appendix E). On most of these models no single family strengthens the bound at all, so their marks in Figure 3(a) lie on the vertical axis, and greedy selection misses the best decomposition on all but three of the scored models with a true joint support. Every benchmark instance has a certain zero-marginal group at $\emptyset$, the classical pair of its class, and GCG's detector, which follows the block structure of the matrix, convexifies on the network-design and facility-location instances one block per arc or per facility, a set with no gain on about two thirds of the instances (Table 3). The phenomenon of Figure 1 is therefore not a property of textbook instances.

What the language models know. On the structured templates, GPT-6.1 Sol names a true joint support first on every instance, and Claude Sonnet 5.5 and Grok 4.20 (Reasoning) do so on about two thirds of them (Table 1). On MIPLIB-NL these three configurations are close to each other and to the lookahead order; the two configurations without reasoning fall behind, and for Grok 4.20 the difference between its two variants exceeds sampling variation. GPT-6.1 Sol and Grok 4.20 (Reasoning) never predict a joint support on the assignment templates, which have none, and no configuration finds the hidden group of a program of Theorem 1(a) more often than chance. The language models and the lookahead are right on partly different models: each configuration with reasoning names the true group first on a few MIPLIB-NL models where the first set of the lookahead order is wrong, and the reverse occurs about as often (Table 12). Most misses of the lookahead are the superset effect of Proposition 1, a true group with further families. On the network-design templates the lookahead at the HiGHS dual is zero on every pair, including the true one, the dependence on the dual that Proposition 1 allows for polyhedral groups, while GPT-6.1 Sol names the true pair first on each of them; with five optimal duals the lookahead names the true group first on every template, and on MIPLIB-NL its hit rate does not change (Table 13).

Where the predictions come from. On the templates the predictions follow the names: GPT-6.1 Sol finds no joint support once the names are permuted or anonymized, and permuting the names lowers the recall of Grok 4.20 (Reasoning) and Claude Sonnet 5.5 beyond sampling variation (Figure 3(b)). MIPLIB-NL names are less standard, and there permuting them lowers no recall beyond sampling variation (Figure 3(c)), so these predictions do not rest on the name attached to each family, while withholding the names and the text together still lowers the recall of both Grok 4.20 variants. Grok 4.20 (Reasoning) names a true group first about twice as often as Grok 4.20 (Non-Reasoning) on both sets of instances, and on MIPLIB-NL its higher hit rate and recall persist when the names are anonymized (Table 10).

What the predictions are worth. On the templates and on MIPLIB-NL every order other than the random one needs about three evaluations or fewer on average, because most of these models have at most four families, and among the orders of Table 2, JointDW with GPT-6.1 Sol is the cheapest on MIPLIB-NL for every s. JointDW needs fewer evaluations than the covering order beyond sampling variation for $s=2$ with every configuration except Grok 4.20 (Non-Reasoning), and the interleaved order does so for every configuration and every s, saving tens of seconds of solve

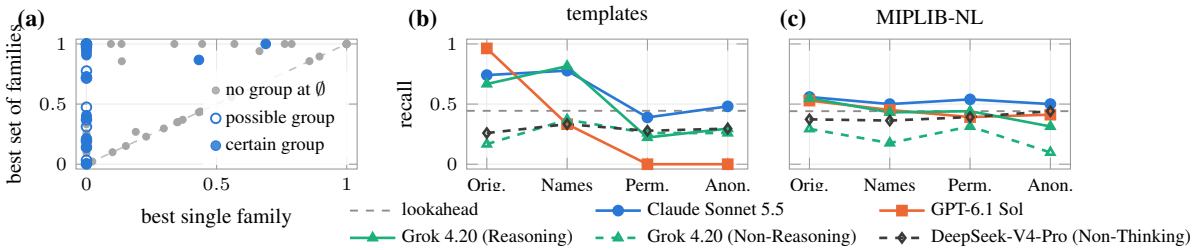


Figure 3. (a) Census of MIPLIB-NL: for each model with a positive gain, the largest gain of a single family against the largest gain of a set of families, both as shares of the LP gap; marks on the vertical axis have no family that strengthens the bound alone. Blue marks have a zero-marginal group at $\emptyset$. (b, c) Recall of joint supports under the four presentations, with the recall of the lookahead order as a dashed line. Solid lines: configurations with reasoning; dashed lines: without.

time per model (Table 25). On MIPLIB-NL no configuration differs from the lookahead order at the HiGHS dual beyond sampling variation, while on the templates the interleaved orders of the two configurations without reasoning cost about half an evaluation more than it. A better dual source helps every configuration: with five optimal duals the lookahead order needs close to one evaluation for $s = 2$ on MIPLIB-NL and exactly one on every template, JointDW with GPT-6.1 Sol does not differ from it beyond sampling variation, and interleaving brings every configuration within one evaluation of it (Table 13). GCG's own decompositions, used as anchors of the staged search, are no better than the covering order on the rich models (Appendix E and Table 15). Asked for ranked groups with probabilities, or for the families that matter, the models give lists that save no evaluations beyond sampling variation when interleaved with the lookahead order, and their stated probabilities exceed the observed hit rates (Appendix F).

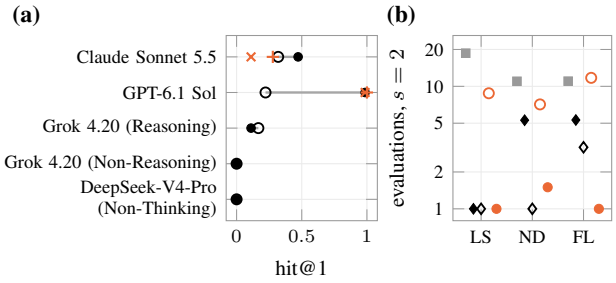


Figure 4. Rich models. (a) hit@1 given the text (open circles) and every coefficient (filled circles), and, for two configurations, every coefficient with the names permuted ($\times$) or replaced by labels ($+$). (b) Mean evaluations until a best set is evaluated at $s = 2$ for lot sizing (LS), network design (ND) and facility location (FL): covering order (gray squares), lookahead order at the HiGHS dual (filled diamonds) and with five duals (open), JointDW with GPT-6.1 Sol given the text (open circles) and every coefficient (filled). Every configuration and $s = 3$: Table 15.

Rich models. On the rich models a covering of all pairs needs more than ten evaluations in every class (Figure 4). The lookahead order finds a best set with its first set of three families on every instance, and for pairs it fails only where every pair has a lookahead value of zero; five duals remove this failure on network design and reduce it on facility location. Given the text, no configuration names a true group first in more than a third of its answers: GPT-6.1 Sol mostly returns no group and states that the coefficients are missing, Claude Sonnet 5.5 often names another classical pair, Grok 4.20 (Reasoning) adds further families to the group, and the two configurations without reasoning name pairs that are not groups. Given every coefficient, GPT-6.1 Sol names the classical group of each class in all but one answer: balance and setup rows, flow balance and arc capacity, and assignment and capacity. That group is true on every instance, and JointDW then needs close to one evaluation for $s = 2$, against nearly four for the lookahead order at the HiGHS dual and close to two with five duals, while no other configuration improves beyond sampling variation. This prediction does not rest on the names: with the family names permuted among the families, or replaced by labels, GPT-6.1 Sol still names the true group first in all but one answer, the pair that carries the classical names after the permutation is never a true group and is never named first, and the hit rate of Claude Sonnet 5.5 falls to about a quarter of its value under the permutation (Table 17). Since all network-design and facility-location instances share their best pair, this measures whether a model commits to the right classical group among distracting families. For sets of three families the lookahead order alone is better.

A predictor trained on labeled evaluations. The learned predictor scores a set from structural features of its decomposition and is trained on the gain tables of other instances, so each training instance costs every evaluation in its table. Trained on the seven other rich models of its class, it needs about as many evaluations for $s = 2$ as JointDW with GPT-6.1 Sol given the coefficients, within sampling variation, and its labels cost more than two hundred evaluations per class (Table 19). Trained on the other two classes and on MIPLIB-NL instead, it names the best pair first on a third of

Table 3. Benchmark instances: share of instances on which the first set of a method is the true best pair, for lot sizing (LS), network design (ND), facility location (FL) and all 59 instances, and the branch-and-price time of the method's set relative to that of the true best pair, as the geometric mean of $(t+1)/(t^*+1)$ in seconds, with the standard error of the bootstrap on the logarithmic scale carried to the ratio. JointDW uses the original presentation. Standard errors, times, gaps and intervals: Appendix F.

	first set is the best pair				time
Method	LS	ND	FL	all	ratio
Lookahead, HiGHS dual	1.00	0.35	0.68	0.68	1.97 ± 0.75
Lookahead, five duals	1.00	0.35	0.95	0.76	1.66 ± 0.56
GCG's detector	1.00	0.00	0.00	0.34	2.11 ± 0.74
JointDW, GPT-6.1 Sol	1.00	0.67	1.00	0.89	1.47 ± 0.42
JointDW, Claude Sonnet 5.5	1.00	0.75	0.68	0.81	1.64 ± 0.56

the rich models and needs about five evaluations more than JointDW, beyond sampling variation; with the lookahead value as a feature it is within sampling variation of JointDW with GPT-6.1 Sol and of the five-dual lookahead order. On MIPLIB-NL, trained on the gain tables of the other scored models, the predictor with the lookahead feature matches the five-dual lookahead order, at more than eight hundred labeled evaluations per held-out model. The language model needs no labels.

Benchmark instances and solve time. The benchmark instances have three families each, so the question is what the first choice costs once branch-and-price runs on it (Table 3). On network design the lookahead names the best pair, flow balance with arc capacity, on about a third of the instances with one dual or with five, because the duals at hand give a larger value to flow balance with arc linking or to arc capacity alone, while the two language models name it on most instances; on facility location GPT-6.1 Sol names assignment with capacity on every instance. Pooled over the instances, both language models name the best pair more often than the detector, and GPT-6.1 Sol more often than the lookahead at the HiGHS dual, beyond sampling variation; against the five-dual lookahead the differences are within sampling variation (Table 23). With every coefficient shown instead of the family tables, prompts of tens to hundreds of thousands of tokens, both configurations name the best pair less often, and the first set of GPT-6.1 Sol still gains more of the LP gap, beyond sampling variation, than that of the lookahead at the HiGHS dual and that of the detector (Table 24). In branch-and-price, the sets of GPT-6.1 Sol come closest to the time of the true best pair, and on network design a wrong pair costs minutes where the right one costs seconds.

7. Discussion

Three sources of side information. A detector that enumerates subsets of constraint classes and keeps the best bound, as GCG's class-based detector does, pays the covering number of Theorem 1 in the worst case, and a detector that follows the block structure of the matrix convexifies, on two of the three benchmark classes, a set without gain on all but one instance. A detector therefore needs information from outside the bounds, which Theorem 2 prices by its number of misses. The LP dual, the text of the model and labeled evaluations of similar instances have different blind spots: the dual is the cheapest and, with several optimal duals, at least as good as any configuration on MIPLIB-NL, but on the benchmark network-design instances it values flow balance with arc linking above the pair that closes the gap; the text with its coefficients is strongest where the dual is degenerate; and labeled evaluations buy accuracy only within the class or collection they come from. Because the interleaved order bounds the loss from any source, a detector can combine them, and a more accurate language model, or a better dual, can replace a current one without risk.

Limitations. The oracle model counts evaluations rather than seconds, and its randomized and asymptotic statements use the cardinality budget s. The scored sets of instances are small, so many intervals are wide, and the MIPLIB-NL models with a complete gain table are too small for the asymptotic savings of Theorem 2 (Appendix F covers the larger ones). The rich models contain no group whose families share no variable (Proposition 1). The language models were likely trained on the textbooks, MIPLIB models and benchmark libraries, and the controlled presentations cannot exclude that a model recognizes an instance from its statistics or coefficients.

8. Conclusion

Choosing which constraint families to convexify is hard exactly when families strengthen the bound only together: bound evaluations then reveal little, finding a group costs a covering number of evaluations, and a prediction with e misses reduces the cost to $\Theta(k^e)$ without an asymptotic price in robustness, so the evaluations a prediction saves measure what the predictor knows. Such groups occur in three of ten MIPLIB-NL models whose bound can be improved and in every benchmark instance we examined. The strongest language models read the text about as well as the dual predicts; one of them names the classical group of a larger model from its coefficients where the dual is degenerate, also with the names permuted or removed; and none needs labeled evaluations. Because the search turns every avoided miss into fewer evaluations and bounds the loss from a wrong prediction, more accurate language models translate into cheaper choices of decompositions.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning and of mathematical optimization. Better choices of decompositions can shorten the solution of planning and scheduling models, and we see no specific negative societal consequence that must be highlighted. We keep every request to and response of the commercial language models, so that the results can be checked without new calls.

References

Aardal, K., Pochet, Y., and Wolsey, L. A. Capacitated facility location: Valid inequalities and facets. *Mathematics of Operations Research*, 20(3):562–582, 1995. doi: 10.1287/moor.20.3.562.

AhmadiTeshnizi, A., Gao, W., Brunborg, H., Talaei, S., Lawless, C., and Udell, M. OptiMUS-0.3: Using large language models to model and solve optimization problems at scale. *arXiv preprint arXiv:2407.19633*, 2024.

Angluin, D. and Chen, J. Learning a hidden hypergraph. *Journal of Machine Learning Research*, 7:2215–2236, 2006.

Atamtürk, A. On capacitated network design cut-set polyhedra. *Mathematical Programming*, 92(3):425–437, 2002. doi: 10.1007/s101070100284.

Baek, J. LLMs can design near-optimal OR algorithms. *arXiv preprint arXiv:2608.27296*, 2026.

Balas, E. Disjunctive programming and a hierarchy of relaxations for discrete optimization problems. *SIAM Journal on Algebraic Discrete Methods*, 6(3):466–486, 1985. doi: 10.1137/0606047.

Banerjee, S., Cohen-Addad, V., Gupta, A., and Li, Z. Graph searching with predictions. *arXiv preprint arXiv:2212.14220*, 2022.

Bárány, I., Van Roy, T. J., and Wolsey, L. A. Uncapacitated lot-sizing: The convex hull of solutions. *Mathematical Programming Study*, 22:32–43, 1984. doi: 10.1007/BFb0121006.

Basso, S. and Ceselli, A. A data driven Dantzig–Wolfe decomposition framework. *Mathematical Programming Computation*, 15(1):153–194, 2023. doi: 10.1007/s12532-022-00230-4.

Basso, S., Ceselli, A., and Tettamanzi, A. Random sampling and machine learning to understand good decompositions. *Annals of Operations Research*, 284(2):501–526, 2020. doi: 10.1007/s10479-018-3067-9.

Bastubbe, M., Lübbecke, M. E., and Witt, J. T. A computational investigation on the strength of Dantzig-Wolfe reformulations. In *17th International Symposium on Experimental Algorithms (SEA 2018)*, volume 103 of *LIPIcs*, pp. 11:1–11:12, 2018. doi: 10.4230/LIPIcs.SEA.2018.11.

Beasley, J. E. OR-Library: Distributing test problems by electronic mail. *Journal of the Operational Research Society*, 41(11):1069–1072, 1990. doi: 10.1057/jors.1990.166.

Bergner, M., Caprara, A., Ceselli, A., Furini, F., Lübbecke, M. E., Malaguti, E., and Traversi, E. Automatic Dantzig–Wolfe reformulation of mixed integer programs. *Mathematical Programming*, 149(1–2):391–424, 2015. doi: 10.1007/s10107-014-0761-5.

Bhattacharyya, S. and Baranwal, M. SOVER: Formal certification of optimization reformulations via LLM-assisted SMT verification. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. To appear; arXiv:2609.00728.

Chen, T. and Guestrin, C. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016. doi: 10.1145/2939672.2939785.

Cheng, S., Ye, T., Deza, A., Qi, X., Li, M., Liu, L., and Guan, C. DecompGym: Benchmarking LLM agents on designing decomposition algorithms. *arXiv preprint arXiv:2610.38175*, 2026.

Crainic, T. G., Frangioni, A., and Gendron, B. Bundle-based relaxation methods for multicommodity capacitated fixed charge network design. *Discrete Applied Mathematics*, 112(1–3):73–99, 2001. doi: 10.1016/S0166-218X(00)00310-3.

Dantzig, G. B. and Wolfe, P. Decomposition principle for linear programs. *Operations Research*, 8(1):101–111, 1960. doi: 10.1287/opre.8.1.101.

Das, A. and Kempe, D. Submodular meets spectral: Greedy algorithms for subset selection, sparse approximation and dictionary selection. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, pp. 1057–1064, 2011.

Erlebach, T., de Lima, M. S., Megow, N., and Schlöter, J. Learning-augmented query policies. *arXiv preprint arXiv:2011.07385*, 2020.

Fort, M. K. and Hedlund, G. A. Minimal coverings of pairs by triples. *Pacific Journal of Mathematics*, 8(4):709–719, 1958. doi: 10.2140/pjm.1958.8.709.

Gamrath, G. and Lübbecke, M. E. Experiments with a generic Dantzig-Wolfe decomposition for integer programs. In *Experimental Algorithms (SEA 2010)*, volume 6049 of *Lecture Notes in Computer Science*, pp. 239–252. Springer, 2010. doi: 10.1007/978-3-642-13193-6_21.

Gandikota, V., Grigorescu, E., Jaggi, S., and Zhou, S. Nearly optimal sparse group testing. *IEEE Transactions on Information Theory*, 65(5):2760–2773, 2019. doi: 10.1109/TIT.2019.2891651.

GCG developers. GCG 3.5 documentation: Constraint class detector. https://gcg.or.rwth-aachen.de/doc-3.5.0/det-consclass.html, 2026a. Accessed 30 October 2026.

GCG developers. GCG documentation: Detection scores. https://gcg.or.rwth-aachen.de/doc/detection-scores.html, 2026b. Accessed 30 October 2026.

Geoffrion, A. M. Lagrangean relaxation for integer programming. *Mathematical Programming Study*, 2:82–114, 1974. doi: 10.1007/BFb0120690.

Gordon, D. M., Kuperberg, G., and Patashnik, O. New constructions for covering designs. *Journal of Combinatorial Designs*, 3(4):269–284, 1995. doi: 10.1002/jcd.3180030404.

Guignard, M. and Kim, S. Lagrangean decomposition: A model yielding stronger Lagrangean bounds. *Mathematical Programming*, 39(2):215–228, 1987. doi: 10.1007/BF02592954.

Hojny, C., Besançon, M., Bestuzheva, K., Borst, S., Dionísio, J., Ehls, J., Eifler, L., Ghannam, M., Gleixner, A., et al. The SCIP optimization suite 10.0. *arXiv preprint arXiv:2511.18580*, 2025.

Huang, C., Tang, Z., Hu, S., Jiang, R., Zheng, X., Ge, D., Wang, B., and Wang, Z. ORLM: A customizable framework in training large models for automated optimization modeling. *Operations Research*, 73(6):2986–3009, 2025. doi: 10.1287/opre.2024.1233.

Kong, M., Jiang, C., Qu, A., Ouyang, W., Zeng, Z., Guo, X., et al. FrontierOR: Benchmarking LLMs' capacity for efficient algorithm design in large-scale optimization. *arXiv preprint arXiv:2605.25246*, 2026.

Kruber, M., Lübbecke, M. E., and Parmentier, A. Learning when to use a decomposition. In *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2017)*, volume 10335 of *Lecture Notes in Computer Science*, pp. 202–210. Springer, 2017. doi: 10.1007/978-3-319-59776-8_16.

Lawless, C., Li, Y., Wikum, A., Udell, M., and Vitercik, E. LLMs for cold-start cutting plane separator configuration. *arXiv preprint arXiv:2412.12038*, 2024.

Le, T. Q., Nguyen, A. T., and Nguyen, V. A. Provably data-driven Lagrangian relaxation for mixed integer linear programming. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2605.19052.

Li, Z., Lu, H., Wei, T., Chen, Y., Liu, W., Lan, Y., Zhang, F., and Wen, Z. Constructing industrial-scale optimization modeling benchmark. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.10450.

Lian, J. J., Chen, H., Qin, H., and Teo, C.-P. Admission without answers: Label-free certification and experience learning for LLM-based optimization modeling. *arXiv preprint arXiv:2608.15565*, 2026a.

Lian, J. J., Sun, Y., Chen, H., Zhang, C., Qin, H., and Teo, C.-P. ReLoop: Structured modeling and behavioral verification for reliable LLM-based optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026b. arXiv:2602.15983.

Liang, K., Lu, Y., Mao, J., Sun, S., Yang, C., Zeng, C., Jin, X., Qin, H., Zhu, R., and Teo, C.-P. Large-scale optimization model auto-formulation: Harnessing LLM flexibility via structured workflow. *arXiv preprint arXiv:2601.09635*, 2026.

Lovász, L. On the ratio of optimal integral and fractional covers. *Discrete Mathematics*, 13(4):383–390, 1975. doi: 10.1016/0012-365X(75)90058-8.

Lübbecke, M. E. and Witt, J. T. The strength of Dantzig–Wolfe reformulations for the stable set and related problems. *Discrete Optimization*, 30:168–187, 2018. doi: 10.1016/j.disopt.2018.07.001.

Mitzenmacher, M. and Vassilvitskii, S. Algorithms with predictions. *Communications of the ACM*, 65(7):33–35, 2022. doi: 10.1145/3528087.

Nie, J., Feng, K., Zhang, X., Yin, S., Wang, Z., and Dong, B. MILP-Evo: Closed-loop fully automatic design of MILP solvers. *arXiv preprint arXiv:2607.18252*, 2026.

Raack, C., Koster, A. M. C. A., Orlowski, S., and Wessäly, R. On cut-based inequalities for capacitated network design polyhedra. *Networks*, 57(2):141–156, 2011. doi: 10.1002/net.20395.

Robbins, H., Lawless, C., Udell, M., and Vitercik, E. FLARE: Verifying MILP reformulations with LLM-based theorem proving. *arXiv preprint arXiv:2608.25220*, 2026.

Rödl, V. On a packing and covering problem. *European Journal of Combinatorics*, 6(1):69–78, 1985. doi: 10.1016/S0195-6698(85)80023-8.

Schönheim, J. On coverings. *Pacific Journal of Mathematics*, 14(4):1405–1411, 1964. doi: 10.2140/pjm.1964.14.1405.

Stein, S. K. Two combinatorial covering theorems. *Journal of Combinatorial Theory, Series A*, 16(3):391–397, 1974. doi: 10.1016/0097-3165(74)90062-4.

Trigeiro, W. W., Thomas, L. J., and McClain, J. O. Capacitated lot sizing with setup times. *Management Science*, 35(3):353–366, 1989. doi: 10.1287/mnsc.35.3.353.

Vanderbeck, F. and Wolsey, L. A. Reformulation and decomposition of integer programs. In *50 Years of Integer Programming 1958–2008*, pp. 431–502. Springer, 2010. doi: 10.1007/978-3-540-68279-0_13.

Witt, J. T. *Polyhedral Aspects of Dantzig-Wolfe Reformulation*. PhD thesis, RWTH Aachen University, 2020.

Xu, L., Zhou, Y., and Pokutta, S. Agentic MIP research: Accelerated constraint handler generation. *arXiv preprint arXiv:2605.09186*, 2026.

Yazdani, M., Mostajabdaveh, M., Aref, S., and Zhou, Z. LLM-guided evolution of symbolic acceleration inequality families for MILP. *Transactions on Machine Learning Research*, 2026. arXiv:2508.11850.

Yuan, H., Peng, J., Bi, J., Zhang, N., Song, S., and Cao, Z. FormuEvo: LLM-guided evolution for discovering solver-efficient mixed-integer programming formulations. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2026. To appear; arXiv:2608.23353.

Zhai, H., Lawless, C., Vitercik, E., and Leqi, L. EquivaMap: Leveraging LLMs for automatic equivalence checking of optimization formulations. *arXiv preprint arXiv:2502.14760*, 2025.

Zhu, R., Cao, M., Zhou, C., Lin, J., and Ge, D. Beyond verified answers: Solver-informed self-distillation for bootstrapping operations research language models. *arXiv preprint arXiv:2609.09957*, 2026.

A. Proofs

Throughout, LP $= \{x : A_f x \geq b_f \text{ for } f \in F,\ \ell \leq x \leq u\}$, and for multipliers $y \geq 0$ on the rows of $F \setminus T$,

$$L_T(y) = y\, b_{F\setminus T} + \min\{(c - yA_{F\setminus T})x : x \in X_T\}.$$

Lemma 1 (monotonicity)**.** *If $T \subseteq T'$ then $P_{T'} \subseteq P_T$. Hence g is nondecreasing and $0 \leq g(T) \leq z_{\mathrm{IP}} - z_{\mathrm{LP}}$.*

Proof. $X_{T'} \subseteq X_T$, so $\mathrm{conv}(X_{T'}) \subseteq \mathrm{conv}(X_T)$, and $P_F = \mathrm{conv}(X)$. □

Lemma 2 (inheritance)**.** *$z(T) = \max_{y\geq 0} L_T(y)$. For $T \subseteq T'$ and every $y \geq 0$ on the rows of $F \setminus T$, $L_{T'}(y|_{F\setminus T'}) \geq L_T(y)$. Solving the blocks of T' once at optimal multipliers of T therefore gives a valid bound of at least $z(T)$.*

Proof. The first statement is Lagrangian duality over the polytope $\mathrm{conv}(X_T)$, on which a linear objective attains its minimum over X_T. For $x \in X_{T'}$ the rows of $T' \setminus T$ hold, so $y_{T'\setminus T}(A_{T'\setminus T}x - b_{T'\setminus T}) \geq 0$. This gives

$$(c - y_{F\setminus T'}A_{F\setminus T'})x + y_{F\setminus T'}b_{F\setminus T'} \geq (c - yA_{F\setminus T})x + y\, b_{F\setminus T}.$$

Minimize the left side over $X_{T'}$ and the right side over $X_T \supseteq X_{T'}$. □

Proposition 2 (support view)**.** *For every $T \subseteq F$, P_T is the set of points of* LP *that satisfy every inequality $\alpha x \geq \beta$ valid for* $\mathrm{conv}(X)$ *that has a family support $S \subseteq T$.*

Proof. An inequality with a family support $S \subseteq T$ is valid for $P_S \supseteq P_T$ by Lemma 1. Conversely, $\mathrm{conv}(X_T)$ is the intersection of its valid inequalities. Each of them is valid for $\mathrm{conv}(X) \subseteq \mathrm{conv}(X_T)$ and for P_T, so it has a family support inside T. □

Polyhedral groups. Let W be a polyhedral zero-marginal group at B. Since $Q_W \subseteq P_B$, $\min\{cx : x \in Q_W\} = z(B) < z(B \cup W)$, so a point $\bar{x} \in Q_W$ with $c\bar{x} = z(B)$ does not lie in $P_{B\cup W}$. By Proposition 2 some valid inequality with a family support inside $B \cup W$ is violated by $\bar{x}$. It has no family support inside any $B \cup S$ with $S \subsetneq W$, since it would then be valid for $P_{B\cup S} \supseteq Q_W$. Every family support of it inside $B \cup W$ therefore contains W. At $B = \emptyset$ the point $\bar{x}$ is an optimum of the LP relaxation, so W is a joint support of an inequality that cuts off an LP optimum.

Interleaving. Let O_1 and O_2 be two query orders and let the interleaved order take their entries alternately, skipping sets already queried. The first t entries of either order appear among the first $2t$ entries of the interleaved order. If O_i queries a set of maximal gain at position t_i, the interleaved order does so within $2\min(t_1, t_2)$ queries.

Proof of Theorem 1. (a) Let the cycle have length $2m + 1 \geq \max(3, w)$ and let its edges be split into w nonempty paths, one for each family of W; each other family is a path on its own vertices. If $W \not\subseteq T$, the rows of T form a forest, whose edge-vertex incidence matrix is totally unimodular, so $\mathrm{conv}(X_T)$ equals the LP relaxation of X_T and $P_T =$ LP. If $W \subseteq T$, the cycle variables range over the stable-set polytope of the cycle, on which $\max \sum_v x_v = m$, against the LP value $(2m + 1)/2$. The path components are integral in both cases and share no variable with the cycle, so $g(T) = \frac{1}{2}[W \subseteq T]$.

(b) On g_W a query returns a nonzero value exactly when it contains W, and the maximal gain $\frac{1}{2}$ is attained exactly by these queries. Against the answer 0 to every query, a deterministic method produces a fixed sequence $T_1, T_2, \ldots$. If $T_1, \ldots, T_{N-1}$ with $N = N(\mathcal{T}, w)$ leave uncovered some w-set W that lies in a member of $\mathcal{T}$, then on g_W the method has not found a maximizer after $N - 1$ queries. Conversely, querying a minimum covering finds every such W. For the randomized bound let W be uniform over the w-subsets and $J = \binom{k}{w}/\binom{s}{w}$. The first j queries of a deterministic method contain at most $j\binom{s}{w}$ of the $\binom{k}{w}$ targets, so its number of queries N_W satisfies $\Pr[N_W > j] \geq 1 - j/J$ and

$$\mathbb{E}N_W = \sum_{j\geq 0} \Pr[N_W > j] \geq \sum_{j=0}^{\lfloor J\rfloor} (1 - j/J) \geq (\lfloor J\rfloor + 1)/2 > J/2.$$

A randomized method is a distribution over deterministic ones, so its expected cost on the uniform target exceeds $J/2$, and the maximum over W is at least this average.

(c) Each s-set contains $\binom{s}{w}$ of the $\binom{k}{w}$ sets of size w, which gives the lower bound. The uniform fractional covering has value $\binom{k}{w}/\binom{s}{w}$, and the greedy covering is within a factor $1+\ln\binom{s}{w}$ of the fractional optimum (Lovász, 1975; Stein, 1974). Rödl's theorem (Rödl, 1985) gives the asymptotics. For unknown w, the additional terms are $\sum_{w'<w} \mathrm{C}(k,s,w') = O(k^{w-1})$.

(d) Every single addition has marginal gain 0 on g_W because $w \geq 2$. □

Proof of Theorem 2. (a) A hidden group with $|W \setminus P| = e$ has $W \cap P$ inside some A and $W \setminus P$ inside some R of stage e, and $|A \cup R| \leq a + (s-a) = s$. Stages 0 to e together cost at most the stated sum.

(b) The hidden groups consistent with $|W| = w$ and $|W \setminus P| = e$ number $\binom{w}{e}\binom{k-w}{e}$. A query T with $a = |T \cap P|$ and $|T \setminus P| \leq s-a$ contains at most $f_e(a)$ of them, and none if $a \notin A_e$. The adversary argument of Theorem 1(b) gives L_e. The sequence f_e is log-concave in a as a product of log-concave sequences, and

$$\frac{f_e(w)}{f_e(w-1)} = \frac{w\,(s-w+1-e)}{e\,(s-w+1)} \geq 1 \iff e(s+1) \leq w(s-w+1).$$

Since e is an integer and $e(s+1) \leq w(s-w+1) < (s+1)(s-w+1)$, the condition implies $e \leq s-w$, so $w \in A_e$, and the maximum is at $a = w$, where $f_e(w) = \binom{w}{e}\binom{s-w}{e}$. For $e = w$, $f_w(a) = \binom{s-a}{w}$ decreases in a, and the maximum is at $a = 0$.

(c) For each fixed a, Rödl's theorem gives $\mathrm{C}(k-w, s-a, e') = (1+o(1))\binom{k-w}{e'}/\binom{s-a}{e'} = \Theta(k^{e'})$, so the sum in (a) is $(1+o(1))$ times its last term. The last term equals $(1+o(1))\binom{k-w}{e}\min_{a\in A_e} c_e(a)$ with $c_e(a) = \mathrm{C}(w,a,w-e)/\binom{s-a}{e}$. Counting gives $\mathrm{C}(w,a,w-e) \geq \binom{w}{e}/\binom{a}{w-e}$, hence $c_e(a) \geq \binom{w}{e}/f_e(a)$, and $\rho_e = \min_a c_e(a) \cdot \max_a f_e(a)/\binom{w}{e} \geq 1$. Under the condition of (b), $c_e(w) = 1/\binom{s-w}{e} = \binom{w}{e}/\max_a f_e(a)$, so $\rho_e = 1$. For $e = w$, $c_w(0) = 1/\binom{s}{w}$ and $\rho_w = 1$, and for $e = 0$, $c_0(w) = 1 = f_0(w)$. Finally, stage w with $a = 0$ is a covering of the w-subsets of $F \setminus P$ by s-sets, so it costs at most $\mathrm{C}(k-w, s, w) \leq \mathrm{C}(k,s,w)$, and the earlier stages cost $O(k^{w-1})$. □

Finite k. With $w = 2$, $s = 3$ and $k = 20$, an entirely wrong prediction costs at most $1 + 18 + \mathrm{C}(18,3,2) = 73$ queries with minimum coverings, against $\mathrm{C}(20,3,2) = 67$ without a prediction (Fort & Hedlund, 1958). Table 4 lists the values plotted in Figure 2(b). The staged search uses greedy coverings. Its value for e misses is the number of queries in stages 0 to e, an upper bound on its worst case, since a group can already lie in a query of an earlier stage. Replaying the orders shows that the bound is attained for k from 9 to 16, except for $k = 11$ with $e = 2$, where the worst case is 14; at $k = 8$ with $e = 3$ the worst case is 4. The greedy covering without prediction was computed up to $k = 20$. For $k = 16$ and $e = 3$, the staged search needs 56 queries against 43 for the greedy covering without prediction. The earlier stages cause this overhead, and the interleaved order of JointDW caps the cost at twice the smaller of the two.

Table 4. Evaluations for $w = 3$ and $s = 6$: for each number of misses e, the queries in stages 0 to e of the staged search with greedy coverings, an upper bound on its worst case, and the lower bound L_e; without prediction, the worst case of the greedy covering and the bound $\binom{k}{3}/\binom{6}{3}$. n.c.: not computed.

	$e=1$		$e=2$		$e=3$		no prediction	
k	staged	L_1	staged	L_2	staged	L_3	greedy	bound
8	3	1.7	6	2.5	7	0.5	4	2.8
12	4	3.0	18	9.0	27	4.2	15	11.0
16	6	4.3	32	19.5	56	14.3	43	28.0
20	7	5.7	57	34.0	112	34.0	85	57.0
24	8	7.0	71	52.5	174	66.5	n.c.	101.2

The split in stage e'. For $(w,s,e) = (4,7,3)$ the condition of Theorem 2(b) fails, $A_3 = \{1,2,3,4\}$, and $c_3(a) = \mathrm{C}(4,a,1)/\binom{7-a}{3}$ takes the values $4/20$, $2/10$, $2/4$ and $1/1$ for $a = 1,2,3,4$. Keeping all of P, which is $a = 4$, is therefore five times costlier than the best split. For $(w,s,e) = (3,6,2)$, the case of Figure 2, $\rho_2 = \frac{3}{10} \cdot 12/3 = 6/5$.

Lemma 3 (one-pass lookahead)**.** *Let y be an optimal dual solution of the LP relaxation for the rows of F, with the bounds $\ell \leq x \leq u$ kept, and let $\ell_y(T) = L_T(y|_{F\setminus T}) - z_{\mathrm{LP}}$. Then $\ell_y(\emptyset) = 0$, $\ell_y(T) \leq g(T)$, ℓ_y is nondecreasing, and $\ell_y(T_1 \cup T_2) = \ell_y(T_1) + \ell_y(T_2)$ whenever no variable appears in both a row of T_1 and a row of T_2. Consequently $\ell_y(W) = 0$ for every zero-marginal group W at $\emptyset$ of order at least two whose families pairwise share no variable, and every such group is LP-geometric.*

Proof. LP duality with the bounds kept gives $L_\emptyset(y) = z_{\mathrm{LP}}$. Weak duality in Lemma 2 gives $L_T(y|_{F\setminus T}) \le z(T)$, and Lemma 2 gives monotonicity. Write $d = c - yA$. Then $L_T(y|_{F\setminus T}) = yb - y_T b_T + \min\{dx + y_T A_T x : x \in X_T\}$. If T_1 and T_2 share no variable, the minimum splits into three parts: one over the variables of T_1, one over the variables of T_2, and one over the remaining variables at their bounds. Subtracting $L_\emptyset(y)$ leaves the sum of the two increments. For the last claim, $|W| \ge 2$ makes every $\{f\}$ a proper subset of W, so $0 = \ell_y(\emptyset) \le \ell_y(W) = \sum_{f\in W} \ell_y(\{f\}) \le \sum_{f\in W} g(\{f\}) = 0$. The rows of different families of W constrain disjoint sets of variables, and the convex hull of a product of sets is the product of their convex hulls, so $\mathrm{conv}(X_W) = \bigcap_{f\in W} \mathrm{conv}(X_{\{f\}})$. Hence $P_W = \bigcap_{f\in W} P_{\{f\}} \supseteq Q_W \supseteq P_W$, and $\min\{cx : x \in Q_W\} = z(W) > z(\emptyset)$, so W is LP-geometric. □

Greedy selection. Let γ be the submodularity ratio of Das & Kempe (2011) with respect to the set U chosen by greedy selection. It is the minimum of $\sum_{x\in S}[g(L\cup\{x\}) - g(L)]/[g(L\cup S) - g(L)]$ over $L \subseteq U$ and disjoint S with $|S| \le s$ and $g(L\cup S) > g(L)$. A zero-marginal group W at $\emptyset$ with $2 \le |W| \le s$ gives $\gamma = 0$ through $L = \emptyset$ and $S = W$.

B. Exact Computations and Examples

Method. For every T, the polytope P_T is written as one LP on the variables of the MILP and, where needed, auxiliary variables. Each block of T is represented either by the convex hull of its integer points, enumerated over the bounds, or by Balas' extended formulation of the union of the polyhedra of the continuous variables over the integer assignments (Balas, 1985). HiGHS solves the LP, the optimal basis is recomputed in rational arithmetic with FLINT through python-flint, and primal and dual feasibility are checked exactly. The kind of a group is computed from Definition 2 with one hull formulation for each $P_{W\setminus\{f\}}$ on the same variables.

Example 1 (LP-geometric groups)**.** Consider $\min\{-y : y \le x_1+x_2,\ y \le \frac{1}{2},\ 2x_1 \le 1,\ 2x_2 \le 1,\ x \in \{0,1\}^2,\ 0 \le y \le 1\}$ with four rows as four families, and $z_{\mathrm{LP}} = -\frac{1}{2}$. Convexifying $2x_1 \le 1$ fixes $x_1 = 0$, and the LP moves the fractional weight to x_2. Each bound row alone therefore has gain 0, while the two bound rows together have gain $\frac{1}{2}$. Each bound row together with both y-rows forms a zero-marginal group of order three with gain $\frac{1}{4}$. The exact computation classifies all three groups as LP-geometric.

Lookahead on the instance of Figure 1. For the pair of capacity and setup rows, whose gain is $51/10$, the one-pass lookahead is 0 at the optimal dual returned by HiGHS and about 2.7 at the optimal dual that maximizes it, in a floating-point computation.

Example 2 (single-item lot sizing)**.** For single-item uncapacitated lot sizing over three periods with demands $(3, 2, 4)$, holding costs 1 and setup costs 8, $z_{\mathrm{LP}} = 40/3$. Each family alone has gain 0, and balance with setup has gain $14/3$ and is polyhedral. Balance and setup rows together imply the (l, S) inequalities (Bárány et al., 1984).

C. Instances

The templates are small instances of five classes with exact gains for every set of families (Table 5). The rich models are larger connected instances of three of these classes with exact gains for every set of at most three families (Table 6).

Lot sizing. Items i and periods t, continuous production x_{it} and stock s_{it}, and binary setups y_{it}. The balance family has rows $s_{i,t-1} + x_{it} - s_{it} = d_{it}$. The setup family has rows $x_{it} \le M_{it} y_{it}$, where M_{it} is the minimum of the capacity of period t and the remaining demand of item i. The capacity family has rows $\sum_i x_{it} \le C_t$. The objective is holding plus setup cost. Demands, capacities, holding costs and setup costs are $((2,3),(2,2))$, $(5,4)$, $(1,1)$ and $(6,6)$ for LS 2×2; $((3,2),(1,3))$, $(5,4)$, $(1,2)$ and $(7,5)$ for LS 2×2 b; and $((2,1),(1,2),(2,2))$, $(6,4)$, $(1,1,1)$ and $(6,5,7)$ for LS 3×2.

Network design. Two commodities travel on the directed arcs $(1,2)$, $(1,3)$, $(2,3)$, $(2,4)$ and $(3,4)$ of a four-node network. Commodity q has continuous flows $x_{qa} \in [0, d_q]$, and arc a has a binary opening variable y_a. The flow-balance family has one conservation row per commodity and node, the arc-linking family has rows $x_{qa} \le d_q y_a$, and the arc-capacity family has rows $\sum_q x_{qa} \le u_a y_a$. The objective is the fixed cost of the opened arcs plus one unit of cost per unit of flow and arc. In ND a the commodities go from node 1 to node 4 with demand 2 and from node 1 to node 3 with demand 1, every capacity is 2, and the fixed costs are $(4,4,3,4,3)$. ND b has the fixed costs $(6,6,5,6,5)$ and is otherwise equal to ND a. In ND c the commodities go from node 1 to node 4 with demand 2 and from node 2 to node 4 with demand 1, the capacities are $(3,2,2,2,3)$ and the fixed costs are $(5,4,2,4,5)$.

Table 5. Instances with exact ground truth. LP gap: $(z_{\mathrm{IP}} - z_{\mathrm{LP}})/|z_{\mathrm{IP}}|$. Gap closed: the share of the LP gap that convexifying the group closes, computed in exact rational arithmetic and rounded. Every group of order two or three is polyhedral. The generalized assignment instances have no joint support, and their best decomposition convexifies the capacity family alone.

Class and families	Instance	k	LP gap (%)	Zero-marginal group	Gap closed (%)
Lot sizing	LS 2×2	3	32.4	BS	63
B balance, S setup, C capacity				CS	63
	LS 2×2 b	3	34.6	BS	60
				CS	57
	LS 3×2	3	22.0	BS	100
				CS	17
Network design	ND a	3	14.7	NU	100
N flow balance, L arc linking, U arc capacity	ND b	3	15.2	NU	100
	ND c	3	5.9	NU	100
Facility location	FL 2×3	3	2.4	AC	100
A assignment, C capacity, L linking	FL 3×3	3	1.8	AC	100
				AL	25
	FL 2×4	3	1.7	AC	100
Generalized assignment	GAP 2×3	2	60.0	none	
A assignment, C capacity	GAP 2×3 b	2	50.9	none	
Hidden group	HG a	4	8.3	BC	100
conflict families A to F	HG b	5	8.3	AD	100
	HG c	6	7.1	CE	100
	HG d	5	16.7	ACE	100

Facility location. Continuous $x_{ij} \in [0, 1]$ is the share of the demand d_j of customer j served by facility i, and binary y_i opens facility i. The assignment family has rows $\sum_i x_{ij} = 1$, the capacity family has rows $\sum_j d_j x_{ij} \le C_i y_i$, and the linking family has rows $x_{ij} \le y_i$. The objective is the opening cost plus $\sum_{ij} c_{ij} d_j x_{ij}$. Demands, capacities, opening costs and unit costs are $(2, 2, 3)$, $(4, 4)$, $(5, 6)$ and $((1, 3, 2), (3, 1, 2))$ for FL 2×3; $(2, 3, 2)$, $(5, 4, 4)$, $(6, 5, 5)$ and $((1, 2, 3), (3, 1, 2), (2, 3, 1))$ for FL 3×3; and $(3, 2, 2, 1)$, $(5, 5)$, $(7, 6)$ and $((1, 2, 3, 2), (3, 2, 1, 1))$ for FL 2×4.

Generalized assignment. Binary x_{ij} assigns job j to machine i. The assignment family has rows $\sum_i x_{ij} = 1$, the capacity family has rows $\sum_j a_{ij} x_{ij} \le b_i$, and the objective is the assignment cost. Weights, capacities and costs are $((5, 5, 4), (5, 5, 4))$, $(9, 9)$ and $((1, 1, 10), (10, 10, 1))$ for GAP 2×3, and $((4, 5, 3), (5, 4, 3))$, $(8, 7)$ and $((2, 1, 9), (9, 8, 1))$ for GAP 2×3 b.

Hidden group. These are the programs of Theorem 1(a) with the objective $\max \sum_v x_v$. In HG a, with $k = 4$, an odd cycle of length 5 is split into two paths, the hidden families B and C. In HG b and HG c, with $k = 5$ and $k = 6$, a triangle is split into two paths, the hidden families A and D, and C and E. In HG d, with $k = 5$, a triangle is split into its three edges, the hidden families A, C and E. The other families are paths of the same lengths on new vertices. Every family has a name of the form Conflict followed by a letter, and all families have the same one-sentence description.

Rich models. Each rich model is a single connected program whose families are standard modeling features on shared variables, with data small enough that every set of at most three families can be evaluated exactly with the method of Appendix B. The lot-sizing models have two products, three periods, six binary setups and the families Balance, SetupLink, Capacity with setup times and overtime, MinLot, StorageCap, SafetyStock, MaxSetupsPerPeriod, OvertimeLimit and SetupBudget. The network-design models have two commodities, four nodes and seven arcs or five nodes and eight arcs, one binary per arc, and the families FlowBalance, ArcLink, ArcCapacity, BuildBudget, NodeOutDegree, OppositeArcs and InArcCover. The facility-location models have three facilities with binary opening variables, three or four customers with split demand, and the families Assign, Capacity, Link, OpenBudget, OpenCount, MinUtilization and FacilityConflict. Data are drawn at random until eight draws per class are accepted. A draw is rejected if it is infeasible, if its LP gap is below two percent, or if a single family closes more than half of the best gain among sets of at most three families. Table 6 summarizes the accepted instances. Every zero-marginal group is polyhedral and connected through shared variables, and in every instance the best set of at most three families closes the whole LP gap.

Table 6. Rich models: number of families k, size, LP gap, the minimal best sets among the sets of at most three families with the number of instances in parentheses, two lot-sizing instances having two such sets, and the number of zero-marginal groups per instance. B: Balance, S: SetupLink, C: Capacity, T: StorageCap, Y: SafetyStock; F: FlowBalance, U: ArcCapacity; A: Assign, P: Capacity.

Class	k	rows × columns	LP gap (%)	best sets	groups
Lot sizing	9	35×21	15.2–51.7	BS (4), BCS (3), BST (1), BSY (2)	2–4
Network design	7	37×21, 43×24	2.8–30.1	FU (8)	1
Facility location	7	21×12, 25×15	5.5–28.5	AP (8)	1–2

Benchmark instances. The benchmark instances use the published formulation and data of each class, with no added family and no generated parameter (Table 7). Lot sizing uses the capacitated instances with setup times of Trigeiro et al. (1989), with the families balance, setup linking and capacity with setup times, and no overtime; the formulation was cross-checked against two independent copies of the data, and the 20 smallest usable instances by items times periods were kept. Network design uses the R instances of multicommodity capacitated fixed-charge network design (Crainic et al., 2001), with the families flow balance, arc linking with $x_{qa} \le \min(d_q, u_a)\, y_a$ and arc capacity. Facility location uses the capacitated instances cap41 to cap134 of the OR-Library (Beasley, 1990), with split demand and the families assignment, capacity and linking $x_{ij} \le y_i$. For the last two classes, every instance whose LP is feasible and whose LP gap against the SCIP optimum is positive was kept, up to the 20 smallest by number of variables; 19 of the 37 facility-location instances have a positive LP gap. Every instance has a proved optimum from SCIP with one thread, seed 0 and a limit of one hour. The ground truth evaluates every set of one or two families with GCG at the root node under the settings of Appendix E. Every instance has a certain zero-marginal group at $\emptyset$, and in every instance the best pair is the classical pair of its class: balance with setup linking, flow balance with arc capacity, and assignment with capacity. Because the original presentation shows no coefficients, instances of one class with the same family tables share a prompt: the 59 instances give eight distinct prompts, one for lot sizing, four for network design and three for facility location.

Table 7. Benchmark instances: number of instances, size, LP gap as $(z_{\mathrm{IP}} - z_{\mathrm{LP}})/|z_{\mathrm{IP}}|$, and the best pair of families, which is a certain zero-marginal group at $\emptyset$ in every instance.

Class	Data	Instances	rows × columns	LP gap (%)	Best pair
Lot sizing	Trigeiro et al. (1989)	20	195×270	63.9–74.5	balance, setup linking
Network design	R set (Crainic et al., 2001)	20	485×385 to 1160×910	0.05–10.7	flow balance, arc capacity
Facility location	OR-Library cap (Beasley, 1990)	19	866×816 to 2600×2550	0.006–0.42	assignment, capacity

D. Prompt and Model Settings

Prompt. Every request consists of one system message and one user message. The user message defines convexification, a family that strengthens the bound alone, a joint support and the integrality property, and explains how a reformulation is specified. It then gives the description, the formulation, the model statistics and the two family tables, with no bound or optimal value. Sections of a description that mention an optimal or best-known value are removed. In the presentations names only, permuted and anonymized, the description and the formulation are replaced by the word withheld. In the permuted and anonymized presentations, the family names in the tables are permuted among the families or replaced by $F_1, \ldots, F_k$ and $V_1, \ldots, V_m$. The system message is

```
You analyze mixed-integer linear programs and design Dantzig-Wolfe reformulations. You answer with one JSON
  object and nothing else.
```

and the user message, with each inserted field written as a name in double angle brackets, is

```
## Task
The mixed-integer linear program (MILP) below is correct as given. You receive its natural-language
  description and formulation when available, and the constraint families and variable families of the solver
  model. Answer three questions about its constraint families, then propose Dantzig-Wolfe (DW) reformulations.

Definitions. Convexifying a set T of constraint families means replacing the LP relaxation by its intersection
  with the convex hull of the integer points that satisfy the constraints of T (with the variable bounds and
  integrality). This is the bound of a DW reformulation whose blocks are the constraints of T.
1. A family "strengthens alone" if convexifying that family by itself gives a bound strictly better than the
  LP relaxation for this model and objective.
2. A "joint support" is a set of two or more families that together strengthen the bound while no proper
  subset of them does: some valid inequalities that cut off LP solutions can only be derived from all of these
  families together.
3. A family "has the integrality property" if convexifying it alone can never improve on the LP relaxation,
```

```
  for any objective.

A reformulation is specified by `master_constraint_families` (kept in the master) and
  `linking_variable_families` (variable families copied into each block, with the copies equated in the
  master; leave empty unless intended). Every constraint not in the master is convexified; two constraints are
  in the same block when they share a variable that is not a linking variable; each block is a pricing problem
  solved by a MIP solver at every column-generation iteration, so a block that contains most of the model is
  as hard as the model itself. Proposals are solved at the root node without cutting planes with a time limit
  of <<time_limit>> seconds and scored by the dual bound at the end of the root node.

## Problem description
<<description>>

## Formulation
<<formulation>>

## Model statistics
rows <<num_rows>>, columns <<num_columns>>, objective: <<sense>>

## Constraint families
Columns: family | rows | sense counts | MIPLIB constraint types | row length min/median/max | variable
  families touched (nonzeros) | all coefficients equal to 1
<<constraint_table>>

## Variable families
Columns: family | columns | types (B binary, I integer, C continuous) | columns with nonzero objective
  coefficient
<<variable_table>>

## Output
Return only a JSON object of this form:
{
 "families": {"<constraint family>": {"integrality_property": "has | lacks | unclear", "strengthens_alone":
   "yes | no | unclear"}},
 "joint_supports": [
  {"families": ["...", "..."], "inequality_class": "<name of the valid inequalities they generate, or
    'unknown'>", "reason": "<at most 40 words>"}
 ],
 "assessment": "<at most 100 words>",
 "decompose": true,
 "candidates": [
  {"master_constraint_families": ["..."],
    "linking_variable_families": [],
    "block_problem": "<one pricing problem, in words>",
    "expected_pricing_difficulty": "easy | moderate | hard"}
 ]
}
List every constraint family under "families". List joint supports most important first (an empty list if you
  expect none). Give up to <<k>> candidates, best first, all different. Set "decompose" to false if you expect
  that no reformulation will beat the LP bound within the time limit; still list your best candidates. Use
  family names exactly as listed (a variable family written as `name:B`, `name:I` or `name:C` contains only
  the binary, integer or continuous columns of that name pattern). Every candidate must leave at least one
  constraint family out of the master, and must either keep at least one constraint family in the master or
  name at least one linking variable family.
```

The time limit is 120 seconds and the number of candidates is three. For the template LS 2×2 in the original presentation, the inserted part of the user message reads

```
## Problem description
2 products are made on one machine over 2 periods to meet known demands. Each period the machine has a limited
  production capacity shared by all products. Producing a product in a period requires a setup for that
  product in that period. Minimize setup and holding cost.

## Formulation
# Formulation

Constraint families:
- Balance_#_#: inventory balance of each product in each period
- SetupLink_#_#: a product can be produced in a period only if its setup is paid
- Capacity_#: total production of all products in a period is at most the capacity

## Model statistics
rows 10, columns 12, objective: minimize

## Constraint families
Columns: family | rows | sense counts | MIPLIB constraint types | row length min/median/max | variable
  families touched (nonzeros) | all coefficients equal to 1
Balance_#_# | 4 | =:4 | aggregation:2, continuous:2 | 2/3/3 | Stock_A_#:3, Stock_B_#:3, Produce_A_#:2,
  Produce_B_#:2 | yes
SetupLink_#_# | 4 | <=:4 | variable_bound:4 | 2/2/2 | Setup_A_#:2, Produce_A_#:2, Setup_B_#:2, Produce_B_#:2 |
```

```
  no
Capacity_# | 2 | <=:2 | continuous:2 | 2/2/2 | Produce_A_#:2, Produce_B_#:2 | yes

## Variable families
Columns: family | columns | types (B binary, I integer, C continuous) | columns with nonzero objective
  coefficient
Setup_B_# | 2 | B:2 | 2
Setup_A_# | 2 | B:2 | 2
Produce_A_# | 2 | C:2 | 0
Stock_A_# | 2 | C:2 | 2
Produce_B_# | 2 | C:2 | 0
Stock_B_# | 2 | C:2 | 2
```

Prompt variants. The ranked variant inserts after the list of joint supports in the answer form

```
 "ranked_groups": [
  {"families": ["...", "..."], "probability": 0.0}
 ],
 "relevant_families": ["..."],
```

and after the sentence on joint supports the instructions

```
Under "ranked_groups", list up to 5 different sets of constraint families, most likely first: each set is
  your guess for the smallest set of families whose convexification gives the best bound of any family-level
  DW reformulation of this model, and "probability" is your probability that it is that set. Under
  "relevant_families", list the constraint families that you expect the smallest best set to contain; list
  more families rather than fewer when unsure.
```

The ranked variant was asked three times for each of the 13 templates with a positive gain, the 16 scored MIPLIB-NL models that had a certain joint group when the variant was run, and the 24 rich models. The data variant inserts after the formulation a section headed `## Model with data`. It holds the complete instance in CPLEX LP format, written by SCIP with the row and column names of the family tables and without the header comment that names the instance. On the rich models these prompts have at most about 3300 tokens, and on the benchmark instances between about 12,000 and 230,000 (Table 24). Each variant has its own cache entries, so no answer of the original prompt is reused. The two control presentations with coefficients, data permuted and data anonymized, permute the family names among the families and replace them by $F_1, \ldots, F_k$ and $V_1, \ldots, V_m$ in the family tables and in the model with data alike. The permutation and the label map of an instance are drawn from a seed derived from the instance name, so the eight instances of a class receive eight different permutations and label maps, identical for the two configurations run with these controls; after the permutation the pair that carries the classical names is a true group in no instance.

Parsing of answers. Each family named in an answer is matched to a shown name. A name with a trailing block suffix after a colon, or with index patterns, is reduced to its stem before matching, a stem that matches no shown name or more than one is dropped, and an answer whose JSON object does not parse counts as a failed prediction. The rule was fixed before the sample of record was scored.

Models. The five configurations are Claude Sonnet 5.5 (Anthropic) with no request option set, so that the default thinking of the endpoint applies, and without streaming, GPT-6.1 Sol (OpenAI) with medium reasoning effort, Grok 4.20 (Reasoning) and Grok 4.20 (Non-Reasoning) (xAI), and DeepSeek-V4-Pro (DeepSeek) in non-thinking mode. All calls go through OpenAI-compatible chat endpoints. The responses of GPT-6.1 Sol on the rich models and to the prompt variants carry the snapshot identifier gpt-6.1-sol-2026-09-29, its responses on the templates and MIPLIB-NL and the responses of the other models name the model without a snapshot. Every request asks for a JSON object, and no request sets a temperature or a completion limit. The endpoint of Claude Sonnet 5.5 limits a completion, thinking included, to 20,000 tokens. Of its 1404 answers on the templates and MIPLIB-NL, 1384 reach this limit, and all of them except the one that does not parse contain a complete answer object. The three samples of a cell share the prompt and differ in a sample index that enters only the cache key, so they differ only through the sampling of the model. Instances whose prompts are identical share cache entries, so a cell of one of them can repeat the answer of the same cell of another, and the bootstrap draws such instances together. Before the runs, each configuration answered a short arithmetic question twice, and we checked the returned identifier, the presence or absence of reasoning output and the reported prompt tokens. During the runs every response was checked for the requested identifier, for reasoning output where the configuration reasons, and for prompt tokens consistent with our messages. Requests that the endpoint ended before a complete response arrived were sent again, and the first complete answer was kept. Before the rich models and the prompt variants were run, each configuration was checked again. For the two configurations whose short check differed from the record, three recorded requests were sent

again, and these configurations were run only once the reported prompt tokens equaled the recorded ones. All answers were produced between 30 October and 7 November 2026, and every planned cell has an answer. On the templates and MIPLIB-NL, 31 answers do not parse, 1 of Claude Sonnet 5.5 and 30 of DeepSeek-V4-Pro, and an answer that does not parse counts as a failed prediction.

Reproducibility. The gain tables, the family summaries of the exported MIPLIB-NL models, the script that exports them from MIPLIB-NL, and every cached request and response are released with the code, so every reported answer and table can be reproduced without a new call to a language model. All computations ran on one workstation with a 24-core Intel processor and 48 GB of memory, using SCIP 10 with GCG and HiGHS.

E. MIPLIB-NL Processing and Solver Settings

Each MIPLIB-NL instance comes with a script that builds the model in a solver interface. We run the script in a child process with the optimize call replaced by an export to MPS. Of the 223 instances, 205 export a linear model. Nine scripts fail, three never call the solver, four export a model whose optimum differs from the reference, namely csched008, neos-4333596-skien, p2m2p1m1p0n100 and prod2, and two export an infeasible model, csched007 and csched010. Constraint families are the row names with their digit runs removed. Of the 205 models, 57 have fewer than two families, 34 have more than six and 6 have general constraints, which leaves 108 for the census.

GCG runs through PyGCGOpt 1.0.0b0 on PySCIPOpt 6.0.0 with SCIP 10, with wall-clock timing, one thread, seed 0 and the root node only, and with presolving, separation, primal solution procedures and constraint propagation disabled in the original and in the master problem, so that every bound comes from the relaxation alone. Each evaluation has a time limit of 120 seconds. An evaluation that ends abnormally is repeated twice with the same settings and once with the next seed, and counts as having no bound if all four attempts fail. Variables shared by blocks are declared as linking variables, and the bounds of instances with an integral objective are rounded before comparison. The LP bound comes from SoPlex through SCIP. For graph40-40-1rand and graph40-80-1rand, whose simplex solve did not finish within four hours, it comes from the interior-point method of HiGHS with crossover, which agrees with SoPlex to a relative difference below 10^{-14} on the three largest other models. Without linking variables, convexifying all k families leaves an empty master and solves the MILP itself, so the census and the scoring use the sets of at most $k-1$ families. The lookahead gives each block 60 seconds and each instance 3600 seconds, with propagation disabled in the block problems as well. Its dual reconstruction sets to zero the row duals of the wrong sign on an infinite side when their absolute value is at most the dual feasibility tolerance of HiGHS, and treats larger ones as errors. The five-dual lookahead adds four optimal duals obtained by maximizing random objectives over the optimal face of the dual, with the dual values boxed at ten times one plus the largest absolute value of the HiGHS dual, and takes for each set the largest of the five values; on 90 of the 102 scored models at least one of the four duals differs from the HiGHS dual. Every solver process runs under a memory cap of 5 GB, raised to 10 GB for 59 models that need it, and the caps of all running processes are limited to 40 GB in total; an evaluation whose attempt overlapped memory pressure, or ended through the memory cap, was rerun under this budget, which changed the bound of 102 evaluations and the classification of four models. A recorded bound is discarded when it is not finite, when it lies beyond the reference optimum by more than a tolerance of 10^{-4} times the larger of the LP gap and the absolute optimum, or when it lies beyond a solution value that SCIP found by more than this tolerance, since GCG returns an infinite dual bound when the time limit ends the Farkas phase of column generation and, on noswot and cvs08r139-94, reported a bound beyond a feasible solution. Of the 3122 family-set evaluations of the census, 46 percent converged, 32 percent stopped at the time limit with a valid bound, and 21 percent returned no bound, among them 24 evaluations of 10 models with a discarded bound. Six of the 108 models have no evaluated set with a bound and are not scored: gen-ip054, graph40-20-1rand, graph40-40-1rand, graph40-80-1rand, opm2-z12-s8 and thor50dday. GCG's own detected decompositions, used as anchors of the staged search, need 2.3 to 3.5 evaluations on average to reach a best set on the 11 to 13 models where they exist. The branch-and-price runs on the benchmark instances use GCG with its default settings, one thread, seed 0 and a limit of 1800 seconds on the given reformulation; a run that fails is repeated with the next seed, and a run that fails twice counts as unsolved at the limit with its root bound. The runs of identical reformulations are shared between methods, and 183 of the 196 runs reached optimality.

Reference optima. Gains are reported as shares of the LP gap when MIPLIB-NL gives an optimal value. Our runs show that this value is wrong for four models. For neos17 the LP bound already exceeds it in a minimization, and GCG proves the value 0.15000258 optimal, which we use instead. For gmu-35-50 it lies above the LP bound of a maximization, and we treat the model as having no reference optimum. For b-ball and radiationm18-12-05 an evaluation recorded a feasible solution

better than the reference, and SCIP, with one thread, seed 0 and a limit of one hour, proves the values 3 and 17566 optimal, which we use instead. Of the 102 scored models, 75 have a positive gain under some decomposition, 23 of them with a certain and 26 with a possible zero-marginal group at $\emptyset$. Of the 98 models with a reference optimum and a positive LP gap, the 69 whose bound improves under some decomposition appear in Figure 3(a).

F. Further Results

Table 8 gives recall under every presentation and Table 9 the paired differences between presentations. Table 10 gives the difference between Grok 4.20 (Reasoning) and Grok 4.20 (Non-Reasoning), Tables 11, 25 and 26 the search costs and the paired differences between orders in evaluations and in solve time, Table 12 the instances on which a configuration and the lookahead order are right, and Table 13 the five-dual lookahead order and the orders interleaved with it. The later parts of this appendix give the results on the rich models with both prompt variants and the control presentations, the learned predictor, the benchmark instances, and the MIPLIB-NL models with more than six families. Standard errors and 95 percent percentile intervals come from 10,000 bootstrap resamples with seed 0. The resampled units are groups of instances whose requests are identical in at least one presentation: the 15 templates form 11 groups, the 102 scored MIPLIB-NL models 99 and the 24 rich models 16. Every order that depends on the order of the families, including the tie-breaking of the lookahead order, is evaluated under the same 1000 random orders of the families of each instance and averaged. Lookahead values that differ by at most 10^{-9} count as ties.

Table 8. Recall of joint supports by presentation, mean $\pm$ standard error.

Configuration	Original	Names only	Permuted	Anonymized
Templates				
Claude Sonnet 5.5	0.74 ± 0.07	0.78 ± 0.11	0.39 ± 0.14	0.48 ± 0.14
GPT-6.1 Sol	0.96 ± 0.03	0.33 ± 0.18	0.00 ± 0.00	0.00 ± 0.00
Grok 4.20 (Reasoning)	0.67 ± 0.08	0.81 ± 0.08	0.22 ± 0.07	0.30 ± 0.13
Grok 4.20 (Non-Reasoning)	0.17 ± 0.10	0.37 ± 0.15	0.26 ± 0.07	0.26 ± 0.12
DeepSeek-V4-Pro (Non-Thinking)	0.26 ± 0.12	0.33 ± 0.06	0.28 ± 0.06	0.30 ± 0.13
MIPLIB-NL				
Claude Sonnet 5.5	0.56 ± 0.10	0.50 ± 0.10	0.54 ± 0.11	0.50 ± 0.08
GPT-6.1 Sol	0.53 ± 0.12	0.45 ± 0.12	0.39 ± 0.11	0.41 ± 0.12
Grok 4.20 (Reasoning)	0.55 ± 0.11	0.43 ± 0.10	0.44 ± 0.10	0.31 ± 0.08
Grok 4.20 (Non-Reasoning)	0.29 ± 0.10	0.18 ± 0.07	0.31 ± 0.09	0.10 ± 0.05
DeepSeek-V4-Pro (Non-Thinking)	0.37 ± 0.09	0.36 ± 0.07	0.39 ± 0.10	0.44 ± 0.09

Table 9. Paired differences in recall between presentations, mean $\pm$ standard error. An asterisk marks a difference whose 95 percent bootstrap interval excludes zero.

Configuration	Names only − permuted	Names only − anonymized	Original − anonymized
Templates			
Claude Sonnet 5.5	$0.39 \pm 0.18^*$	$0.30 \pm 0.14^*$	0.26 ± 0.20
GPT-6.1 Sol	$0.33 \pm 0.18^*$	$0.33 \pm 0.18^*$	$0.96 \pm 0.03^*$
Grok 4.20 (Reasoning)	$0.59 \pm 0.06^*$	$0.52 \pm 0.18^*$	$0.37 \pm 0.15^*$
Grok 4.20 (Non-Reasoning)	0.11 ± 0.16	0.11 ± 0.12	-0.09 ± 0.08
DeepSeek-V4-Pro (Non-Thinking)	0.06 ± 0.05	0.04 ± 0.11	-0.04 ± 0.03
MIPLIB-NL			
Claude Sonnet 5.5	-0.04 ± 0.05	0.00 ± 0.07	0.06 ± 0.07
GPT-6.1 Sol	0.06 ± 0.06	0.04 ± 0.06	0.12 ± 0.08
Grok 4.20 (Reasoning)	-0.01 ± 0.04	0.12 ± 0.07	$0.24 \pm 0.07^*$
Grok 4.20 (Non-Reasoning)	$-0.14 \pm 0.07^*$	0.08 ± 0.07	$0.20 \pm 0.09^*$
DeepSeek-V4-Pro (Non-Thinking)	-0.03 ± 0.09	-0.08 ± 0.07	-0.07 ± 0.10

Five optimal duals. Table 13 gives the first set and the search cost of the five-dual lookahead order and of the JointDW orders interleaved with it. On the templates its first set is a true joint support on every instance, against two thirds for the HiGHS dual, and it needs one evaluation on every template. On MIPLIB-NL its first set is unchanged, and its order needs -1.00 ± 0.75 evaluations relative to the HiGHS dual for $s = 2$, within sampling variation. On the rich models it needs 1.73 ± 0.77 evaluations for $s = 2$ against 3.87 ± 1.36 at the HiGHS dual, a difference within sampling variation, and its

Table 10. Grok 4.20 (Reasoning) minus Grok 4.20 (Non-Reasoning), paired over instances, hit@1 and recall, mean ± standard error. An asterisk marks a difference whose 95 percent bootstrap interval excludes zero.

Presentation	hit@1	recall
Templates		
Original	$0.33 \pm 0.13^*$	$0.50 \pm 0.12^*$
Names only	0.37 ± 0.24	0.44 ± 0.23
Permuted	-0.15 ± 0.14	-0.04 ± 0.07
Anonymized	-0.11 ± 0.11	0.04 ± 0.11
MIPLIB-NL		
Original	$0.25 \pm 0.11^*$	$0.25 \pm 0.11^*$
Names only	$0.27 \pm 0.11^*$	$0.25 \pm 0.10^*$
Permuted	0.10 ± 0.13	0.13 ± 0.12
Anonymized	$0.22 \pm 0.11^*$	$0.22 \pm 0.10^*$

Table 11. Number of evaluations until a best decomposition is evaluated, by size limit s, mean ± standard error. JointDW uses the original presentation.

Order	$s = 2$	$s = 3$	$s = k - 1$
Templates			
Random order	4.57 ± 0.77	4.56 ± 0.82	3.82 ± 0.36
Covering order	2.86 ± 0.61	2.33 ± 0.31	1.82 ± 0.07
Lookahead order	2.00 ± 0.76	1.92 ± 0.71	1.92 ± 0.71
JointDW, Claude Sonnet 5.5	2.47 ± 0.67	2.11 ± 0.41	1.71 ± 0.22
JointDW, GPT-6.1 Sol	2.07 ± 0.67	1.72 ± 0.47	1.16 ± 0.14
JointDW, Grok 4.20 (Reasoning)	2.48 ± 0.67	2.10 ± 0.43	1.59 ± 0.18
JointDW, Grok 4.20 (Non-Reasoning)	2.92 ± 0.64	2.68 ± 0.48	2.16 ± 0.23
JointDW, DeepSeek-V4-Pro (Non-Thinking)	3.14 ± 0.55	2.69 ± 0.29	2.14 ± 0.13
MIPLIB-NL			
Random order	4.82 ± 0.59	5.70 ± 0.80	5.96 ± 1.02
Covering order	3.08 ± 0.44	2.84 ± 0.40	2.11 ± 0.11
Lookahead order	2.18 ± 0.78	2.12 ± 0.78	2.29 ± 0.94
JointDW, Claude Sonnet 5.5	2.20 ± 0.34	2.26 ± 0.36	1.91 ± 0.25
JointDW, GPT-6.1 Sol	1.79 ± 0.34	1.99 ± 0.44	1.78 ± 0.31
JointDW, Grok 4.20 (Reasoning)	2.42 ± 0.46	2.32 ± 0.40	1.90 ± 0.30
JointDW, Grok 4.20 (Non-Reasoning)	2.76 ± 0.48	2.75 ± 0.49	2.04 ± 0.25
JointDW, DeepSeek-V4-Pro (Non-Thinking)	2.51 ± 0.34	2.51 ± 0.34	2.08 ± 0.23

Table 12. Instances on which a configuration, in at least two of its three original samples, and the lookahead order name a true joint support first.

Instances	Configuration	both	configuration only	lookahead only	neither
Templates	Claude Sonnet 5.5	6	0	0	3
Templates	GPT-6.1 Sol	6	3	0	0
Templates	Grok 4.20 (Reasoning)	5	0	1	3
Templates	Grok 4.20 (Non-Reasoning)	3	0	3	3
Templates	DeepSeek-V4-Pro (Non-Thinking)	3	0	3	3
MIPLIB-NL	Claude Sonnet 5.5	7	3	2	5
MIPLIB-NL	GPT-6.1 Sol	7	2	2	6
MIPLIB-NL	Grok 4.20 (Reasoning)	7	3	2	5
MIPLIB-NL	Grok 4.20 (Non-Reasoning)	4	1	5	7
MIPLIB-NL	DeepSeek-V4-Pro (Non-Thinking)	3	0	6	8

first set, scored as a joint support, is less often exactly the true group. Against the five-dual order, JointDW with GPT-6.1 Sol does not differ beyond sampling variation on MIPLIB-NL at any s, while JointDW with each other configuration needs more evaluations at every s. Interleaving with the five-dual order costs every configuration at most 0.5 evaluations more than the five-dual order on MIPLIB-NL, and on the templates it costs the four configurations other than GPT-6.1 Sol between 0.4 and 0.9 evaluations more. On the rich models, interleaving with the five-dual order lowers the cost of interleaving with the HiGHS dual by about 40 percent for $s = 2$.

Table 13. Five-dual lookahead. Top: hit@1 and recall of the first set of the lookahead order at the HiGHS dual and with five optimal duals, mean ± standard error, on the nine structured templates, the 17 MIPLIB-NL models with a certain joint group and the 24 rich models. Bottom: number of evaluations until a best set is evaluated, by size limit s, for the two lookahead orders and for JointDW interleaved with the five-dual order, on the 13 templates with a positive gain, the 17 MIPLIB-NL models and the 24 rich models; JointDW uses the original presentation, and on the rich models the text variant.

	Templates		MIPLIB-NL		Rich models	
First set	hit@1	recall	hit@1	recall	hit@1	recall
Lookahead, HiGHS dual	0.67 ± 0.24	0.44 ± 0.17	0.53 ± 0.12	0.44 ± 0.11	0.21 ± 0.09	0.19 ± 0.09
Lookahead, five duals	1.00 ± 0.00	0.78 ± 0.11	0.53 ± 0.12	0.44 ± 0.11	0.04 ± 0.04	0.04 ± 0.04

	Templates			MIPLIB-NL			Rich models	
Order	$s = 2$	3	$k - 1$	2	3	$k - 1$	2	3
Lookahead, HiGHS dual	1.92 ± 0.71	1.92 ± 0.71	1.92 ± 0.71	2.18 ± 0.78	2.12 ± 0.78	2.29 ± 0.94	3.87 ± 1.36	1.00 ± 0.00
Lookahead, five duals	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.18 ± 0.13	1.24 ± 0.16	1.65 ± 0.52	1.73 ± 0.77	1.00 ± 0.00
JointDW interleaved with the five-dual order								
Claude Sonnet 5.5	1.41 ± 0.14	1.45 ± 0.14	1.50 ± 0.16	1.56 ± 0.14	1.63 ± 0.13	1.72 ± 0.21	2.15 ± 0.54	1.65 ± 0.07
GPT-6.1 Sol	1.19 ± 0.11	1.18 ± 0.11	1.10 ± 0.08	1.34 ± 0.15	1.33 ± 0.10	1.33 ± 0.10	2.43 ± 0.76	1.73 ± 0.04
Grok 4.20 (Reasoning)	1.40 ± 0.12	1.42 ± 0.13	1.43 ± 0.14	1.51 ± 0.15	1.60 ± 0.17	1.66 ± 0.22	2.11 ± 0.45	1.48 ± 0.07
Grok 4.20 (Non-Reasoning)	1.68 ± 0.16	1.74 ± 0.15	1.84 ± 0.17	1.61 ± 0.16	1.69 ± 0.17	1.96 ± 0.41	2.57 ± 0.60	2.00 ± 0.00
DeepSeek-V4-Pro (Non-Thinking)	1.81 ± 0.09	1.84 ± 0.07	1.89 ± 0.11	1.66 ± 0.13	1.74 ± 0.11	2.02 ± 0.37	2.58 ± 0.62	1.99 ± 0.01

Rich models, prompt variants and control presentations. Tables 14 and 15 give the predictions and search costs on the rich models with and without coefficients. With every coefficient, JointDW with GPT-6.1 Sol minus the lookahead order at the HiGHS dual is -2.70 ± 1.33 evaluations for $s = 2$ and 1.10 ± 0.46 for $s = 3$, both beyond sampling variation. Against the five-dual lookahead order it is -0.56 ± 0.74 for $s = 2$ and 1.10 ± 0.46 for $s = 3$, and the interleaved order is -0.71 ± 0.71 and 0.18 ± 0.08; JointDW with Claude Sonnet 5.5 needs 3.96 ± 0.95 evaluations more than the five-dual order for $s = 2$ and its interleaving 0.18 ± 0.32 more, and the three other configurations need between 7.4 and 10.3 more, which interleaving reduces to less than one (Table 18). Without coefficients, GPT-6.1 Sol returns no joint support in 49 of its 72 answers, and with them in one. Table 17 gives the two control presentations with coefficients. For GPT-6.1 Sol, permuting the names changes hit@1 by 0.00 ± 0.02 and lowers recall by 0.07 ± 0.03, beyond sampling variation, and the search cost by less than sampling variation; replacing the names by labels changes neither beyond sampling variation. For Claude Sonnet 5.5, permuting the names lowers hit@1 by 0.36 ± 0.08 and raises the cost of JointDW by 5.06 ± 1.16 evaluations for $s = 2$, both beyond sampling variation, and the labels lower hit@1 by 0.19 ± 0.09. For the ranked variant, coverage@m is the share of samples whose first m groups contain a best set, and the ranked order evaluates the listed groups in turn before the staged search. Table 16 gives the ranked variant. Its lists of relevant families hold on average 52 to 95 percent of the families, depending on the configuration and the set of instances, and contain a best set in 8 to 100 percent of the samples. Evaluating the sets inside the list first, in lookahead order, and interleaving this order with the lookahead order is never cheaper than the lookahead order beyond sampling variation for $s = 2$ and $s = 3$, and it is more expensive in 12 of these 30 comparisons, all on the templates and the rich models. Lists built from the joint supports or the reformulations of the original answers are not cheaper either. Pooled over configurations and instances, the groups stated with a probability in $[0.2, 0.4)$, $[0.4, 0.6)$ and $[0.6, 0.8)$ are true in 8, 11 and 42 percent of the cases.

Learned predictor and combined orders. The learned predictor is a gradient-boosted regression (Chen & Guestrin, 2016) of the gain share of a set of families, the gain divided by the best gain of the instance, from static features of the decomposition that convexifies the set in the manner of Basso & Ceselli (2023): numbers and sizes of the blocks, their densities, the size of the border, statistics of the right-hand sides and the objective coefficients, the share of integer variables, the number of variables shared by the families of the set, and, in the variant with lookahead, the lookahead value of the set at the HiGHS dual. The hyperparameters are those of Basso & Ceselli (2023): depth 13, learning rate 0.1, 100 rounds, seed 0. The sets of at most s families are ordered by decreasing predicted share, and the order is scored like the lookahead order: as

Table 14. Rich models: hit@1 and recall given the text and given every coefficient, mean ± standard error over 24 instances.

	hit@1		recall	
Configuration	text	coefficients	text	coefficients
Claude Sonnet 5.5	0.32 ± 0.06	0.47 ± 0.08	0.44 ± 0.11	0.50 ± 0.06
GPT-6.1 Sol	0.22 ± 0.05	0.99 ± 0.01	0.24 ± 0.08	0.91 ± 0.03
Grok 4.20 (Reasoning)	0.17 ± 0.07	0.11 ± 0.04	0.17 ± 0.07	0.08 ± 0.03
Grok 4.20 (Non-Reasoning)	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
DeepSeek-V4-Pro (Non-Thinking)	0.00 ± 0.00	0.00 ± 0.00	0.02 ± 0.02	0.01 ± 0.01

Table 15. Rich models: number of evaluations until a best set is evaluated, mean ± standard error over 24 instances. The GCG anchor exists on the lot-sizing and facility-location models.

	$s = 2$		$s = 3$	
Order	text	coefficients	text	coefficients
Random order	17.33 ± 1.31		17.53 ± 4.31	
Covering order	13.56 ± 1.19		10.45 ± 2.94	
Lookahead order	3.87 ± 1.36		1.00 ± 0.00	
GCG anchor, 16 instances	15.23 ± 1.72		15.80 ± 4.23	
JointDW				
Claude Sonnet 5.5	8.12 ± 1.00	5.69 ± 0.69	7.47 ± 1.77	4.53 ± 0.52
GPT-6.1 Sol	9.21 ± 0.55	1.17 ± 0.16	6.92 ± 0.87	2.10 ± 0.46
Grok 4.20 (Reasoning)	8.75 ± 0.86	9.17 ± 0.46	6.34 ± 1.52	6.16 ± 1.12
Grok 4.20 (Non-Reasoning)	10.68 ± 0.62	10.27 ± 0.40	12.56 ± 3.45	12.19 ± 2.87
DeepSeek-V4-Pro (Non-Thinking)	11.93 ± 0.91	12.00 ± 0.82	10.93 ± 2.66	11.38 ± 2.51
Interleaved with lookahead				
Claude Sonnet 5.5	3.46 ± 0.91	3.39 ± 0.90	1.65 ± 0.07	1.63 ± 0.08
GPT-6.1 Sol	3.90 ± 1.12	1.01 ± 0.01	1.73 ± 0.04	1.18 ± 0.08
Grok 4.20 (Reasoning)	3.53 ± 0.86	3.36 ± 0.78	1.48 ± 0.07	1.58 ± 0.06
Grok 4.20 (Non-Reasoning)	4.40 ± 1.12	4.30 ± 1.05	2.00 ± 0.00	2.00 ± 0.00
DeepSeek-V4-Pro (Non-Thinking)	4.27 ± 1.08	4.29 ± 1.05	1.99 ± 0.01	1.94 ± 0.02

Table 16. Ranked variant: coverage of the first one and three groups for $s = 2$, and the ranked order interleaved with the lookahead order minus the lookahead order, in evaluations, mean ± standard error. An asterisk marks a difference whose 95 percent bootstrap interval excludes zero.

Configuration	coverage@1	coverage@3	$s = 2$	$s = 3$
MIPLIB-NL				
Claude Sonnet 5.5	0.48 ± 0.11	0.73 ± 0.10	0.01 ± 0.39	-0.73 ± 0.86
GPT-6.1 Sol	0.77 ± 0.10	0.94 ± 0.06	-0.89 ± 0.83	-0.81 ± 0.72
Grok 4.20 (Reasoning)	0.50 ± 0.11	0.65 ± 0.11	-0.27 ± 0.54	-0.67 ± 0.83
Grok 4.20 (Non-Reasoning)	0.40 ± 0.11	0.77 ± 0.10	0.19 ± 0.38	0.15 ± 0.35
DeepSeek-V4-Pro (Non-Thinking)	0.35 ± 0.10	0.83 ± 0.07	0.11 ± 0.40	-0.15 ± 0.56
Rich models				
Claude Sonnet 5.5	0.24 ± 0.10	0.43 ± 0.09	-0.60 ± 0.69	$0.59 \pm 0.09^*$
GPT-6.1 Sol	0.24 ± 0.06	0.24 ± 0.06	-0.33 ± 0.61	$0.57 \pm 0.11^*$
Grok 4.20 (Reasoning)	0.01 ± 0.01	0.01 ± 0.01	0.60 ± 0.25	$0.69 \pm 0.07^*$
Grok 4.20 (Non-Reasoning)	0.00 ± 0.00	0.00 ± 0.00	$0.81 \pm 0.11^*$	$0.94 \pm 0.04^*$
DeepSeek-V4-Pro (Non-Thinking)	0.00 ± 0.00	0.01 ± 0.01	0.15 ± 0.39	$0.89 \pm 0.04^*$

Table 17. Control presentations with every coefficient on the 24 rich models. Top: hit@1 and recall, mean ± standard error, when the family names are shown as they are (data), permuted among the families (permuted) or replaced by labels (anonymized), pooled and by class. Bottom: number of evaluations until a best set is evaluated for JointDW and for its interleaving with the lookahead order at the HiGHS dual, pooled over the 24 instances.

	data		permuted		anonymized	
Configuration	hit@1	recall	hit@1	recall	hit@1	recall
Pooled						
GPT-6.1 Sol	0.99 ± 0.01	0.91 ± 0.03	0.99 ± 0.01	0.83 ± 0.04	1.00 ± 0.00	0.86 ± 0.04
Claude Sonnet 5.5	0.47 ± 0.08	0.50 ± 0.06	0.11 ± 0.04	0.15 ± 0.05	0.28 ± 0.06	0.40 ± 0.05
Lot sizing						
GPT-6.1 Sol	1.00 ± 0.00	0.84 ± 0.06	1.00 ± 0.00	0.71 ± 0.07	1.00 ± 0.00	0.75 ± 0.07
Claude Sonnet 5.5	0.83 ± 0.06	0.33 ± 0.06	0.21 ± 0.08	0.11 ± 0.04	0.42 ± 0.11	0.21 ± 0.06
Network design						
GPT-6.1 Sol	0.96 ± 0.04	0.96 ± 0.04	0.96 ± 0.04	0.96 ± 0.04	1.00 ± 0.00	1.00 ± 0.00
Claude Sonnet 5.5	0.17 ± 0.08	0.71 ± 0.07	0.04 ± 0.04	0.21 ± 0.06	0.00 ± 0.00	0.50 ± 0.10
Facility location						
GPT-6.1 Sol	1.00 ± 0.00	0.92 ± 0.04	1.00 ± 0.00	0.83 ± 0.08	1.00 ± 0.00	0.83 ± 0.08
Claude Sonnet 5.5	0.42 ± 0.13	0.48 ± 0.10	0.08 ± 0.05	0.15 ± 0.11	0.42 ± 0.08	0.48 ± 0.07

Order	data	permuted	anonymized
$s = 2$			
GPT-6.1 Sol, JointDW	1.17 ± 0.16	1.27 ± 0.18	1.00 ± 0.00
GPT-6.1 Sol, interleaved	1.01 ± 0.01	1.03 ± 0.02	1.00 ± 0.00
Claude Sonnet 5.5, JointDW	5.69 ± 0.70	10.75 ± 0.79	8.45 ± 0.63
Claude Sonnet 5.5, interleaved	3.39 ± 0.90	4.29 ± 1.12	3.25 ± 0.82
$s = 3$			
GPT-6.1 Sol, JointDW	2.10 ± 0.46	2.14 ± 0.46	1.99 ± 0.46
GPT-6.1 Sol, interleaved	1.18 ± 0.08	1.19 ± 0.08	1.17 ± 0.08
Claude Sonnet 5.5, JointDW	4.53 ± 0.51	10.18 ± 2.04	7.56 ± 1.31
Claude Sonnet 5.5, interleaved	1.62 ± 0.08	1.86 ± 0.05	1.72 ± 0.06

an answer, as a search order, as the anchor of the staged search, and interleaved with the five-dual lookahead order. The labeling cost of a split is the number of evaluations in the gain tables of its training instances, which is what the labels cost. Three splits are reported in Table 19: within a class, trained on the other instances of the class of the test instance; leave one class out, trained on the templates and rich models of the other two classes and on all scored MIPLIB-NL models; and MIPLIB-NL leave one out, trained on the other scored MIPLIB-NL models. Table 20 gives the cost against the number of training instances of the same class, with 20 random draws of the training set for each size. Against JointDW with GPT-6.1 Sol given the coefficients, the structural order within a class differs by -0.06 ± 0.19 evaluations for $s = 2$ and -0.20 ± 0.13 for $s = 3$, and across classes by 4.81 ± 1.43 and 1.09 ± 1.22; with the lookahead feature, the differences across classes are 0.17 ± 0.32 and -0.68 ± 0.31. On MIPLIB-NL, where the structural order is scored on the 88 and 89 scored models with a best set of at most s families, it differs from the five-dual lookahead order by 0.60 ± 0.26 evaluations for $s = 2$ and by 0.05 ± 0.12 with the lookahead feature; against JointDW with GPT-6.1 Sol on the same models, the structural order differs by -0.10 ± 0.22 for $s = 2$ and -0.70 ± 0.37 for $s = 3$, and the order with the lookahead feature by -0.70 ± 0.18, beyond sampling variation, and -0.41 ± 0.41. The five features of largest gain importance are listed with the released data. Two combined orders were also scored: a union-first order that evaluates the union of the first sets of the language model and the five-dual lookahead before the interleaved order, and a stacked predictor whose features add the answer of the language model. On MIPLIB-NL the union-first order needs 1.26 ± 0.06 evaluations for $s = 2$ against 1.36 ± 0.07 for the interleaved order and 1.21 ± 0.09 for the five-dual order on the same 88 models, and on the rich models with coefficients it needs 1.01 against 1.01; the stacked predictor is at most 0.2 evaluations cheaper than the better of its sources and up to 0.75 evaluations more expensive. Neither changes a conclusion, and both are released with the code.

Benchmark instances. Table 21 gives, per class and method, the share of instances on which the first set is the true best pair, the gain of the first set as a share of the LP gap, the number of instances on which it is zero, the branch-and-price time and final gap of the method's set, and the number of instances solved within the limit; the true best pair is included as a reference. On lot sizing every method chooses the same pair; the runs on GCG's own decomposition of these two families end 22 seconds earlier with a final gap larger by 0.0001. GCG's detector convexifies arc linking with arc capacity

Table 18. Rich models with every coefficient: paired differences in the number of evaluations until a best set is evaluated, JointDW and its interleaving with the five-dual lookahead order minus the five-dual lookahead order, for $s = 2$ and $s = 3$, pooled over the 24 instances and by class, under the data presentation and, for two configurations, under the two control presentations. Mean $\pm$ standard error of the cluster bootstrap over prompt groups, with an asterisk when the 95 percent interval excludes zero.

	JointDW minus five-dual		Interleaved minus five-dual	
Configuration, presentation	$s = 2$	$s = 3$	$s = 2$	$s = 3$
Pooled				
Claude Sonnet 5.5, data	$3.96 \pm 0.95^*$	$3.53 \pm 0.51^*$	0.18 ± 0.32	$0.62 \pm 0.08^*$
Claude Sonnet 5.5, data, permuted	$9.02 \pm 1.06^*$	$9.18 \pm 2.04^*$	$0.76 \pm 0.10^*$	$0.86 \pm 0.05^*$
Claude Sonnet 5.5, data, anonymized	$6.72 \pm 1.10^*$	$6.56 \pm 1.31^*$	0.17 ± 0.54	$0.72 \pm 0.06^*$
GPT-6.1 Sol, data	-0.56 ± 0.74	$1.10 \pm 0.46^*$	-0.71 ± 0.71	$0.18 \pm 0.08^*$
GPT-6.1 Sol, data, permuted	-0.46 ± 0.75	$1.14 \pm 0.46^*$	-0.70 ± 0.71	$0.19 \pm 0.08^*$
GPT-6.1 Sol, data, anonymized	-0.73 ± 0.71	$0.99 \pm 0.46^*$	-0.73 ± 0.71	$0.17 \pm 0.08^*$
Grok 4.20 (Reasoning), data	$7.44 \pm 0.91^*$	$5.16 \pm 1.12^*$	0.51 ± 0.24	$0.58 \pm 0.06^*$
Grok 4.20 (Non-Reasoning), data	$8.55 \pm 0.87^*$	$11.19 \pm 2.87^*$	$0.84 \pm 0.16^*$	$1.00 \pm 0.00^*$
DeepSeek-V4-Pro (Non-Thinking), data	$10.27 \pm 1.15^*$	$10.38 \pm 2.52^*$	$0.84 \pm 0.13^*$	$0.94 \pm 0.02^*$
Lot sizing				
Claude Sonnet 5.5, data	$1.43 \pm 0.51^*$	$3.39 \pm 1.32^*$	$0.11 \pm 0.04^*$	$0.46 \pm 0.17^*$
Claude Sonnet 5.5, data, permuted	$11.53 \pm 2.09^*$	$17.52 \pm 4.95^*$	$0.73 \pm 0.09^*$	$0.79 \pm 0.13^*$
Claude Sonnet 5.5, data, anonymized	$7.79 \pm 1.34^*$	$11.36 \pm 3.28^*$	$0.54 \pm 0.10^*$	$0.62 \pm 0.12^*$
GPT-6.1 Sol, data	0.00 ± 0.00	$2.97 \pm 1.09^*$	0.00 ± 0.00	$0.50 \pm 0.18^*$
GPT-6.1 Sol, data, permuted	0.47 ± 0.44	$3.23 \pm 1.03^*$	0.04 ± 0.04	$0.54 \pm 0.17^*$
GPT-6.1 Sol, data, anonymized	0.00 ± 0.00	$2.97 \pm 1.09^*$	0.00 ± 0.00	$0.50 \pm 0.18^*$
Grok 4.20 (Reasoning), data	$9.38 \pm 0.82^*$	$9.12 \pm 2.80^*$	$0.67 \pm 0.05^*$	$0.62 \pm 0.14^*$
Grok 4.20 (Non-Reasoning), data	$11.85 \pm 0.27^*$	$24.32 \pm 6.50^*$	$1.00 \pm 0.00^*$	$1.00 \pm 0.00^*$
DeepSeek-V4-Pro (Non-Thinking), data	$16.47 \pm 0.46^*$	$22.79 \pm 5.30^*$	$1.00 \pm 0.00^*$	$1.00 \pm 0.00^*$
Network design				
Claude Sonnet 5.5, data	$6.81 \pm 0.69^*$	$3.96 \pm 0.41^*$	$0.83 \pm 0.08^*$	$0.83 \pm 0.08^*$
Claude Sonnet 5.5, data, permuted	$9.58 \pm 0.48^*$	$5.68 \pm 0.28^*$	$0.96 \pm 0.04^*$	$0.96 \pm 0.04^*$
Claude Sonnet 5.5, data, anonymized	$9.36 \pm 0.50^*$	$5.39 \pm 0.35^*$	$0.99 \pm 0.01^*$	$0.96 \pm 0.04^*$
GPT-6.1 Sol, data	0.50 ± 0.47	0.32 ± 0.30	0.04 ± 0.04	0.04 ± 0.04
GPT-6.1 Sol, data, permuted	0.33 ± 0.30	0.18 ± 0.17	0.04 ± 0.04	0.04 ± 0.04
GPT-6.1 Sol, data, anonymized	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Grok 4.20 (Reasoning), data	$6.95 \pm 0.43^*$	$2.64 \pm 0.40^*$	$0.77 \pm 0.05^*$	$0.46 \pm 0.08^*$
Grok 4.20 (Non-Reasoning), data	$8.32 \pm 0.31^*$	$4.85 \pm 0.22^*$	$1.00 \pm 0.00^*$	$1.00 \pm 0.00^*$
DeepSeek-V4-Pro (Non-Thinking), data	$7.98 \pm 0.33^*$	$3.95 \pm 0.31^*$	$0.95 \pm 0.02^*$	$0.86 \pm 0.05^*$
Facility location				
Claude Sonnet 5.5, data	3.65 ± 2.38	$3.25 \pm 0.69^*$	-0.40 ± 0.92	$0.58 \pm 0.13^*$
Claude Sonnet 5.5, data, permuted	$5.97 \pm 1.89^*$	$4.34 \pm 0.33^*$	$0.60 \pm 0.27^*$	$0.83 \pm 0.05^*$
Claude Sonnet 5.5, data, anonymized	3.01 ± 2.48	$2.92 \pm 0.36^*$	-1.02 ± 1.56	$0.58 \pm 0.08^*$
GPT-6.1 Sol, data	-2.18 ± 2.07	0.00 ± 0.00	-2.18 ± 2.07	0.00 ± 0.00
GPT-6.1 Sol, data, permuted	-2.18 ± 2.07	0.00 ± 0.00	-2.18 ± 2.07	0.00 ± 0.00
GPT-6.1 Sol, data, anonymized	-2.18 ± 2.07	0.00 ± 0.00	-2.18 ± 2.07	0.00 ± 0.00
Grok 4.20 (Reasoning), data	$5.99 \pm 2.42^*$	$3.72 \pm 0.57^*$	0.10 ± 0.71	$0.67 \pm 0.09^*$
Grok 4.20 (Non-Reasoning), data	$5.48 \pm 2.05^*$	$4.40 \pm 0.03^*$	0.52 ± 0.46	$1.00 \pm 0.00^*$
DeepSeek-V4-Pro (Non-Thinking), data	$6.37 \pm 2.08^*$	$4.39 \pm 0.09^*$	0.58 ± 0.39	$0.97 \pm 0.01^*$

on network design and capacity with linking on facility location. Table 22 gives the paired differences in time between JointDW and the three baselines, and the time of each method's set relative to the true best pair as the geometric mean of $(t + 1)/(t^* + 1)$ with its interval. Pooled over the 59 instances, JointDW with Claude Sonnet 5.5 saves 93 ± 54 seconds against the single-dual lookahead beyond sampling variation, and no other pooled difference exceeds sampling variation; on facility location both configurations save between 4 and 6 seconds against the detector, and on network design the differences are within sampling variation because a wrong pair on a few instances costs hundreds of seconds. The language models answer eight distinct prompts, and the bootstrap resamples these groups, so the intervals are wide. Table 23 gives the paired differences in hit@1 and in the gain of the first set between JointDW and the three baselines, and between the five-dual lookahead and the detector; on lot sizing every difference is zero. Pooled over the 59 instances, both configurations exceed the detector beyond sampling variation, GPT-6.1 Sol exceeds the lookahead at the HiGHS dual, and neither configuration differs from the five-dual lookahead beyond sampling variation.

Table 19. Learned predictor: hit@1 of its first set and of the five-dual lookahead order, number of evaluations until a best set is evaluated for the learned order and for the staged search anchored at its first set, mean ± standard error, the mean labeling cost of the training set in evaluations, and the number of test instances, by split and variant, for $s = 2$ and $s = 3$. Templates: the instances with a positive gain at s for the within-class split, and the nine structured templates for the leave-one-class-out split.

		hit@1		evaluations			
Instances	s	learned	five duals	learned order	staged	labels	n
within class, structural							
Templates	2	0.81 ± 0.11	0.92 ± 0.08	1.25 ± 0.20	1.21 ± 0.15	19.7	14
Templates	3	0.86 ± 0.10	1.00 ± 0.00	1.33 ± 0.28	1.19 ± 0.14	28.5	15
Rich models	2	0.96 ± 0.03	0.96 ± 0.04	1.11 ± 0.05	2.38 ± 0.64	235.7	24
Rich models	3	0.42 ± 0.13	0.04 ± 0.04	1.90 ± 0.49	3.23 ± 1.22	595.0	24
within class, structural + lookahead							
Templates	2	0.81 ± 0.11	0.92 ± 0.08	1.25 ± 0.20	1.21 ± 0.15	19.7	14
Templates	3	0.86 ± 0.10	1.00 ± 0.00	1.33 ± 0.28	1.19 ± 0.14	28.5	15
Rich models	2	0.96 ± 0.04	0.96 ± 0.04	1.04 ± 0.04	1.32 ± 0.34	235.7	24
Rich models	3	0.33 ± 0.12	0.04 ± 0.04	1.04 ± 0.04	1.65 ± 0.58	595.0	24
leave one class out, structural							
Templates	2	0.44 ± 0.23	1.00 ± 0.00	2.78 ± 0.78	1.94 ± 0.29	1393.7	9
Templates	3	0.89 ± 0.12	1.00 ± 0.00	1.22 ± 0.25	1.17 ± 0.18	2607.0	9
Rich models	2	0.33 ± 0.14	0.96 ± 0.04	5.98 ± 1.38	9.63 ± 2.32	1393.7	24
Rich models	3	0.04 ± 0.04	0.04 ± 0.04	3.19 ± 1.51	4.79 ± 1.26	2607.0	24
leave one class out, structural + lookahead							
Templates	2	0.67 ± 0.24	1.00 ± 0.00	2.33 ± 0.95	1.50 ± 0.36	1393.7	9
Templates	3	0.44 ± 0.22	1.00 ± 0.00	2.78 ± 0.85	1.83 ± 0.33	2607.0	9
Rich models	2	0.96 ± 0.04	0.96 ± 0.04	1.33 ± 0.26	2.25 ± 0.79	1393.7	24
Rich models	3	0.00 ± 0.00	0.04 ± 0.04	1.42 ± 0.22	3.67 ± 1.45	2607.0	24
MIPLIB-NL leave one out, structural							
MIPLIB-NL	2	0.56 ± 0.12	0.65 ± 0.12	1.81 ± 0.25	1.93 ± 0.21	811.0	88
MIPLIB-NL	3	0.47 ± 0.12	0.59 ± 0.12	1.90 ± 0.42	1.64 ± 0.19	1199.1	89
MIPLIB-NL leave one out, structural + lookahead							
MIPLIB-NL	2	0.65 ± 0.12	0.65 ± 0.12	1.26 ± 0.11	1.30 ± 0.14	811.0	88
MIPLIB-NL	3	0.47 ± 0.12	0.59 ± 0.12	2.04 ± 0.51	1.70 ± 0.24	1199.1	89

Table 20. Learned predictor against the number of training instances of the same class on the rich models: number of evaluations until a best set is evaluated for the learned order, mean ± standard error over 20 random draws of the training set, with the labeling cost of each training set in evaluations.

		training instances				
Class	variant, s	1	2	4	7	labels
Facility location	structural, 2	6.05 ± 1.32	3.30 ± 1.14	1.11 ± 0.03	1.05 ± 0.03	28, 56, 112, 196
	structural, 3	1.60 ± 0.14	1.52 ± 0.17	1.10 ± 0.07	1.00 ± 0.00	63, 126, 252, 441
	with lookahead, 2	11.16 ± 1.15	6.78 ± 1.27	4.40 ± 1.21	1.10 ± 0.07	28, 56, 112, 196
	with lookahead, 3	1.06 ± 0.03	1.05 ± 0.04	1.00 ± 0.00	1.00 ± 0.00	63, 126, 252, 441
Lot sizing	structural, 2	1.40 ± 0.02	1.31 ± 0.02	1.25 ± 0.02	1.34 ± 0.06	45, 90, 180, 315
	structural, 3	3.60 ± 0.28	4.05 ± 0.32	4.58 ± 0.53	3.05 ± 0.70	129, 258, 516, 903
	with lookahead, 2	1.19 ± 0.05	1.01 ± 0.01	1.00 ± 0.00	1.00 ± 0.00	45, 90, 180, 315
	with lookahead, 3	1.57 ± 0.17	1.45 ± 0.13	1.82 ± 0.24	1.10 ± 0.07	129, 258, 516, 903
Network design	structural, 2	5.39 ± 1.11	3.84 ± 1.13	1.00 ± 0.00	1.00 ± 0.00	28, 56, 112, 196
	structural, 3	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	63, 126, 252, 441
	with lookahead, 2	5.39 ± 1.11	3.84 ± 1.13	1.00 ± 0.00	1.00 ± 0.00	28, 56, 112, 196
	with lookahead, 3	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	63, 126, 252, 441

Table 21. Benchmark instances, per class and method: share of instances on which the first set is the true best pair, gain of the first set as a share of the LP gap, number of instances on which this gain is zero, branch-and-price time in seconds and final gap of the method's set, mean ± standard error over the instances, and the number of instances solved within 1800 seconds. JointDW uses the original presentation with three samples per instance, averaged within an instance.

Method	first set is best	gain of first set	gain 0	time (s)	final gap	solved
Lot sizing						
True best pair	1.00 ± 0.00	0.964 ± 0.000	0	537.9 ± 0.0	0.00330 ± 0.00000	17/20
Lookahead, HiGHS dual	1.00 ± 0.00	0.964 ± 0.000	0	537.9 ± 0.0	0.00330 ± 0.00000	17/20
Lookahead, five duals	1.00 ± 0.00	0.964 ± 0.000	0	537.9 ± 0.0	0.00330 ± 0.00000	17/20
GCG's detector	1.00 ± 0.00	0.964 ± 0.000	0	515.7 ± 0.0	0.00342 ± 0.00000	17/20
JointDW, GPT-6.1 Sol	1.00 ± 0.00	0.964 ± 0.000	0	537.9 ± 0.0	0.00330 ± 0.00000	17/20
JointDW, Claude Sonnet 5.5	1.00 ± 0.00	0.964 ± 0.000	0	537.9 ± 0.0	0.00330 ± 0.00000	17/20
Network design						
True best pair	1.00 ± 0.00	1.000 ± 0.000	0	2.4 ± 0.9	0.00000 ± 0.00000	20/20
Lookahead, HiGHS dual	0.35 ± 0.12	0.628 ± 0.044	4	280.5 ± 125.7	0.00099 ± 0.00041	17/20
Lookahead, five duals	0.35 ± 0.12	0.642 ± 0.032	3	279.9 ± 125.9	0.00207 ± 0.00080	17/20
GCG's detector	0.00 ± 0.00	0.003 ± 0.004	19	22.6 ± 7.8	0.00000 ± 0.00000	20/20
JointDW, GPT-6.1 Sol	0.67 ± 0.15	0.807 ± 0.096	1	95.5 ± 63.9	0.00095 ± 0.00066	19/20
JointDW, Claude Sonnet 5.5	0.75 ± 0.15	0.827 ± 0.123	1	93.9 ± 75.0	0.00025 ± 0.00021	19/20
Facility location						
True best pair	1.00 ± 0.00	1.000 ± 0.000	0	1.2 ± 0.6	0.00000 ± 0.00000	19/19
Lookahead, HiGHS dual	0.68 ± 0.24	0.684 ± 0.238	6	96.1 ± 90.9	0.00004 ± 0.00004	18/19
Lookahead, five duals	0.95 ± 0.05	0.947 ± 0.051	1	1.3 ± 0.6	0.00000 ± 0.00000	19/19
GCG's detector	0.00 ± 0.00	0.000 ± 0.000	19	7.1 ± 2.7	0.00000 ± 0.00000	19/19
JointDW, GPT-6.1 Sol	1.00 ± 0.00	1.000 ± 0.000	0	1.2 ± 0.6	0.00000 ± 0.00000	19/19
JointDW, Claude Sonnet 5.5	0.68 ± 0.02	0.684 ± 0.017	6	2.6 ± 1.3	0.00000 ± 0.00000	19/19
All instances						
True best pair	1.00 ± 0.00	0.988 ± 0.009	0	183.6 ± 127.1	0.00112 ± 0.00078	56/59
Lookahead, HiGHS dual	0.68 ± 0.15	0.760 ± 0.110	10	308.4 ± 103.4	0.00147 ± 0.00069	52/59
Lookahead, five duals	0.76 ± 0.13	0.849 ± 0.064	4	277.6 ± 112.4	0.00182 ± 0.00069	53/59
GCG's detector	0.34 ± 0.24	0.328 ± 0.228	38	184.8 ± 118.7	0.00116 ± 0.00081	56/59
JointDW, GPT-6.1 Sol	0.89 ± 0.08	0.922 ± 0.043	1	215.1 ± 119.2	0.00144 ± 0.00073	55/59
JointDW, Claude Sonnet 5.5	0.81 ± 0.09	0.827 ± 0.071	7	215.0 ± 120.0	0.00120 ± 0.00076	55/59

Table 22. Benchmark instances. Left: paired differences in branch-and-price time in seconds, JointDW minus a baseline, mean ± standard error, with an asterisk when the 95 percent bootstrap interval excludes zero. Right: time of each method's set relative to the true best pair, as the geometric mean over the instances of $(t+1)/(t^*+1)$, with its 95 percent interval and an asterisk when the interval of the mean log ratio excludes zero, and the difference in final gap.

Configuration	baseline	n	difference (s)
Lot sizing			
JointDW, GPT-6.1 Sol	Lookahead, HiGHS dual	20	0.0 ± 0.0
JointDW, GPT-6.1 Sol	Lookahead, five duals	20	0.0 ± 0.0
JointDW, GPT-6.1 Sol	GCG's detector	20	22.2 ± 0.0*
JointDW, Claude Sonnet 5.5	Lookahead, HiGHS dual	20	0.0 ± 0.0
JointDW, Claude Sonnet 5.5	Lookahead, five duals	20	0.0 ± 0.0
JointDW, Claude Sonnet 5.5	GCG's detector	20	22.2 ± 0.0*
Network design			
JointDW, GPT-6.1 Sol	Lookahead, HiGHS dual	20	−185.0 ± 150.1
JointDW, GPT-6.1 Sol	Lookahead, five duals	20	−184.4 ± 150.4
JointDW, GPT-6.1 Sol	GCG's detector	20	72.8 ± 56.9
JointDW, Claude Sonnet 5.5	Lookahead, HiGHS dual	20	−186.6 ± 66.9
JointDW, Claude Sonnet 5.5	Lookahead, five duals	20	−186.0 ± 66.8
JointDW, Claude Sonnet 5.5	GCG's detector	20	71.2 ± 77.0
Facility location			
JointDW, GPT-6.1 Sol	Lookahead, HiGHS dual	19	−94.8 ± 91.3
JointDW, GPT-6.1 Sol	Lookahead, five duals	19	0.0 ± 0.0
JointDW, GPT-6.1 Sol	GCG's detector	19	−5.9 ± 2.1*
JointDW, Claude Sonnet 5.5	Lookahead, HiGHS dual	19	−93.5 ± 91.6
JointDW, Claude Sonnet 5.5	Lookahead, five duals	19	1.3 ± 0.7*
JointDW, Claude Sonnet 5.5	GCG's detector	19	−4.5 ± 1.4*
All instances			
JointDW, GPT-6.1 Sol	Lookahead, HiGHS dual	59	−93.2 ± 73.6
JointDW, GPT-6.1 Sol	Lookahead, five duals	59	−62.5 ± 66.0
JointDW, GPT-6.1 Sol	GCG's detector	59	30.3 ± 25.6
JointDW, Claude Sonnet 5.5	Lookahead, HiGHS dual	59	−93.3 ± 53.7*
JointDW, Claude Sonnet 5.5	Lookahead, five duals	59	−62.6 ± 44.0
JointDW, Claude Sonnet 5.5	GCG's detector	59	30.2 ± 31.0

Method	n	time ratio	interval	gap difference
Lot sizing				
Lookahead, HiGHS dual	20	1.000	[1.000, 1.000]	0.00000 ± 0.00000
Lookahead, five duals	20	1.000	[1.000, 1.000]	0.00000 ± 0.00000
GCG's detector	20	0.807*	[0.807, 0.807]	0.00012 ± 0.00000*
JointDW, GPT-6.1 Sol	20	1.000	[1.000, 1.000]	0.00000 ± 0.00000
JointDW, Claude Sonnet 5.5	20	1.000	[1.000, 1.000]	0.00000 ± 0.00000
Network design				
Lookahead, HiGHS dual	20	4.733*	[1.460, 11.060]	0.00099 ± 0.00041
Lookahead, five duals	20	4.328*	[1.460, 10.325]	0.00207 ± 0.00080
GCG's detector	20	3.976*	[3.154, 4.953]	0.00000 ± 0.00000
JointDW, GPT-6.1 Sol	20	3.125*	[1.236, 6.659]	0.00095 ± 0.00066
JointDW, Claude Sonnet 5.5	20	3.107	[1.000, 14.356]	0.00025 ± 0.00021
Facility location				
Lookahead, HiGHS dual	19	1.586	[0.998, 5.794]	0.00004 ± 0.00004
Lookahead, five duals	19	1.029	[1.000, 1.115]	0.00000 ± 0.00000
GCG's detector	19	2.982*	[2.666, 3.717]	0.00000 ± 0.00000
JointDW, GPT-6.1 Sol	19	1.000	[1.000, 1.000]	0.00000 ± 0.00000
JointDW, Claude Sonnet 5.5	19	1.403*	[1.215, 1.789]	0.00000 ± 0.00000
All instances				
Lookahead, HiGHS dual	59	1.965*	[1.116, 4.854]	0.00035 ± 0.00024
Lookahead, five duals	59	1.659*	[1.034, 3.817]	0.00070 ± 0.00051
GCG's detector	59	2.111*	[1.224, 3.927]	0.00004 ± 0.00003
JointDW, GPT-6.1 Sol	59	1.471*	[1.023, 3.048]	0.00032 ± 0.00032
JointDW, Claude Sonnet 5.5	59	1.638*	[1.080, 3.877]	0.00008 ± 0.00009

Table 23. Benchmark instances: paired differences in hit@1 and in the gain of the first set as a share of the LP gap, JointDW under the original presentation minus a baseline, and the five-dual lookahead minus GCG's detector, on network design (ND, 20 instances), facility location (FL, 19) and all 59 instances; on lot sizing every difference is zero. Mean ± standard error of the cluster bootstrap over prompt groups, with an asterisk when the 95 percent interval excludes zero.

		difference in hit@1			difference in gain		
Configuration	baseline	ND	FL	all	ND	FL	all
JointDW, GPT-6.1 Sol	Lookahead, HiGHS dual	0.32 ± 0.22	0.32 ± 0.24	$0.21 \pm 0.13^*$	0.18 ± 0.13	0.32 ± 0.24	$0.16 \pm 0.11^*$
JointDW, GPT-6.1 Sol	Lookahead, five duals	0.32 ± 0.22	0.05 ± 0.05	0.12 ± 0.10	0.17 ± 0.12	0.05 ± 0.05	0.07 ± 0.06
JointDW, GPT-6.1 Sol	GCG's detector	$0.67 \pm 0.15^*$	$1.00 \pm 0.00^*$	$0.55 \pm 0.21^*$	$0.80 \pm 0.10^*$	$1.00 \pm 0.00^*$	$0.59 \pm 0.22^*$
JointDW, Claude Sonnet 5.5	Lookahead, HiGHS dual	0.40 ± 0.23	0.00 ± 0.25	0.14 ± 0.14	0.20 ± 0.15	0.00 ± 0.25	0.07 ± 0.11
JointDW, Claude Sonnet 5.5	Lookahead, five duals	0.40 ± 0.23	$-0.26 \pm 0.07^*$	0.05 ± 0.13	0.19 ± 0.15	$-0.26 \pm 0.07^*$	-0.02 ± 0.09
JointDW, Claude Sonnet 5.5	GCG's detector	$0.75 \pm 0.15^*$	$0.68 \pm 0.02^*$	$0.47 \pm 0.18^*$	$0.82 \pm 0.12^*$	$0.68 \pm 0.02^*$	$0.50 \pm 0.19^*$
Lookahead, five duals	GCG's detector	$0.35 \pm 0.12^*$	$0.95 \pm 0.05^*$	$0.42 \pm 0.18^*$	$0.64 \pm 0.03^*$	$0.95 \pm 0.05^*$	$0.52 \pm 0.19^*$

Benchmark instances with every coefficient shown. Table 24 gives JointDW with GPT-6.1 Sol and Claude Sonnet 5.5 on the benchmark instances under the data variant of the prompt, the complete instance in LP format as for the rich models, with three samples per instance. Every instance then has its own prompt, so the bootstrap resamples instances. The median prompt holds 11,988, 45,976 and 70,323 tokens on lot sizing, network design and facility location for GPT-6.1 Sol and 19,870, 76,465 and 114,940 for Claude Sonnet 5.5, as counted by each endpoint, and the longest 141,956 and 230,191. GPT-6.1 Sol returns a joint support in every answer and Claude Sonnet 5.5 in 159 of 177; three answers of Claude Sonnet 5.5 name a set of three families, which the gain tables of these instances, over sets of at most two families, score as a miss without gain. Relative to the original presentation, hit@1 is lower by 0.11 ± 0.04 for GPT-6.1 Sol and by 0.25 ± 0.05 for Claude Sonnet 5.5, both beyond sampling variation and mostly on network design, where hit@1 is 0.40 and 0.17 against 0.67 and 0.75 under the original presentation, and the branch-and-price time of the chosen sets is higher by 132 ± 52 and 151 ± 52 seconds. Against the baselines, the first set of GPT-6.1 Sol gains more of the LP gap than that of the lookahead at the HiGHS dual, by 0.105 ± 0.051, and than that of the detector, by 0.54 ± 0.06, and differs from that of the five-dual lookahead by 0.02 ± 0.04; the first set of Claude Sonnet 5.5 gains more than that of the detector and less than that of the two lookahead orders. The sets chosen under this variant required eight branch-and-price runs that no other method had required; two of them, capacity with setup linking on two lot-sizing instances, reached the time limit.

Models with more than six families. The 34 exported MIPLIB-NL models with more than six families are too large for a complete gain table. Models with more than 26 families are left out, because the lookahead over all sets of at most three families does not fit its time limit, which leaves 30 models with 7 to 26 families. With sets of at most three families and the census settings, each order runs for ten evaluations under one random order of the families. The orders are a greedy covering of all pairs by triples, greedy forward selection, the lookahead order, and three random orders. Evaluated sets are shared between the orders. One model returned no bound for any set (uccase10), 24 returned bounds without a positive gain, and 5 showed a positive gain (bab5, comp08-2idx, comp12-2idx, sct2, supportcase33). Of the 1524 evaluated sets, 592 converged, 680 stopped at the time limit with a bound and 252 returned no bound, mostly through the memory cap, the wall-clock limit or a parameter error inside GCG, and 3 of them through a discarded bound; one set on the greedy path of sct32 was evaluated after the audit of the bounds. On the five models with a gain, we average over $b = 1, \ldots, 10$ the share of the best gain found after b evaluations. It is 0.47 for the lookahead order, 0.33 for random orders, 0.31 for greedy selection and 0.15 for the covering order, with intervals from about 0.01 to 0.86 that do not separate the orders.

Table 24. Benchmark instances with every coefficient shown. Top: per class, hit@1, gain of the first set as a share of the LP gap, branch-and-price time in seconds, final gap and number of instances solved within 1800 seconds for the chosen sets, the time relative to the true best pair as the geometric mean of $(t+1)/(t^*+1)$ with its 95 percent interval, and the number of samples without a joint support; three samples per instance, averaged within an instance, mean $\pm$ standard error of the bootstrap over instances. Bottom: paired differences, JointDW under this variant minus a baseline or minus the same configuration under the original presentation, with an asterisk when the 95 percent interval excludes zero. The baselines are those of Table 21.

Method	hit@1	gain of first set	time (s)	final gap	solved	time ratio	no support
Lot sizing							
JointDW, GPT-6.1 Sol, data	1.00 ± 0.00	0.964 ± 0.005	537.9 ± 134.8	0.00330 ± 0.00192	17.00/20	1.000 [1.000, 1.000]	0/60
JointDW, Claude Sonnet 5.5, data	0.95 ± 0.03	0.934 ± 0.020	586.0 ± 134.6	0.00460 ± 0.00206	16.33/20	1.097 [0.961, 1.308]	1/60
Network design							
JointDW, GPT-6.1 Sol, data	0.40 ± 0.07	0.688 ± 0.049	485.6 ± 164.1	0.00336 ± 0.00165	14.67/20	10.631^* [3.982, 29.541]	0/60
JointDW, Claude Sonnet 5.5, data	0.17 ± 0.06	0.514 ± 0.065	491.3 ± 165.3	0.00276 ± 0.00119	14.67/20	12.794^* [4.846, 34.872]	0/60
Facility location							
JointDW, GPT-6.1 Sol, data	0.95 ± 0.03	0.947 ± 0.028	1.2 ± 0.3	0.00000 ± 0.00000	19.00/19	0.995 [0.986, 1.000]	0/57
JointDW, Claude Sonnet 5.5, data	0.58 ± 0.06	0.596 ± 0.062	3.4 ± 0.8	0.00000 ± 0.00000	19.00/19	1.660^* [1.301, 2.146]	17/57
All instances							
JointDW, GPT-6.1 Sol, data	0.78 ± 0.04	0.865 ± 0.025	347.4 ± 78.6	0.00226 ± 0.00088	50.67/59	2.225^* [1.478, 3.572]	0/177
JointDW, Claude Sonnet 5.5, data	0.56 ± 0.05	0.683 ± 0.039	366.3 ± 79.5	0.00250 ± 0.00084	50.00/59	2.882^* [1.907, 4.596]	18/177

Configuration	baseline	n	difference in hit@1	in gain	in time (s)
Lot sizing					
JointDW, GPT-6.1 Sol, data	Lookahead, HiGHS dual	20	0.00 ± 0.00	0.000 ± 0.000	0.0 ± 0.0
JointDW, GPT-6.1 Sol, data	Lookahead, five duals	20	0.00 ± 0.00	0.000 ± 0.000	0.0 ± 0.0
JointDW, GPT-6.1 Sol, data	GCG's detector	20	0.00 ± 0.00	0.000 ± 0.000	22.2 ± 25.0
JointDW, GPT-6.1 Sol, data	GPT-6.1 Sol, original presentation	20	0.00 ± 0.00	0.000 ± 0.000	0.0 ± 0.0
JointDW, Claude Sonnet 5.5, data	Lookahead, HiGHS dual	20	-0.05 ± 0.03	-0.030 ± 0.020	48.0 ± 34.7
JointDW, Claude Sonnet 5.5, data	Lookahead, five duals	20	-0.05 ± 0.03	-0.030 ± 0.020	48.0 ± 34.7
JointDW, Claude Sonnet 5.5, data	GCG's detector	20	-0.05 ± 0.03	-0.030 ± 0.020	70.2 ± 43.0
JointDW, Claude Sonnet 5.5, data	Claude Sonnet 5.5, original presentation	20	-0.05 ± 0.03	-0.030 ± 0.020	48.0 ± 34.7
Network design					
JointDW, GPT-6.1 Sol, data	Lookahead, HiGHS dual	20	0.05 ± 0.15	0.059 ± 0.099	205.2 ± 134.7
JointDW, GPT-6.1 Sol, data	Lookahead, five duals	20	0.05 ± 0.15	0.046 ± 0.096	205.8 ± 152.4
JointDW, GPT-6.1 Sol, data	GCG's detector	20	$0.40 \pm 0.07^*$	$0.685 \pm 0.049^*$	$463.0 \pm 159.0^*$
JointDW, GPT-6.1 Sol, data	GPT-6.1 Sol, original presentation	20	$-0.27 \pm 0.10^*$	$-0.120 \pm 0.052^*$	$390.2 \pm 134.0^*$
JointDW, Claude Sonnet 5.5, data	Lookahead, HiGHS dual	20	-0.18 ± 0.10	-0.114 ± 0.065	$210.8 \pm 112.9^*$
JointDW, Claude Sonnet 5.5, data	Lookahead, five duals	20	-0.18 ± 0.11	$-0.127 \pm 0.062^*$	211.4 ± 134.5
JointDW, Claude Sonnet 5.5, data	GCG's detector	20	$0.17 \pm 0.06^*$	$0.511 \pm 0.066^*$	$468.6 \pm 161.7^*$
JointDW, Claude Sonnet 5.5, data	Claude Sonnet 5.5, original presentation	20	$-0.58 \pm 0.08^*$	$-0.313 \pm 0.071^*$	$397.4 \pm 133.0^*$
Facility location					
JointDW, GPT-6.1 Sol, data	Lookahead, HiGHS dual	19	$0.26 \pm 0.11^*$	$0.263 \pm 0.110^*$	$-94.8 \pm 92.3^*$
JointDW, GPT-6.1 Sol, data	Lookahead, five duals	19	0.00 ± 0.06	0.000 ± 0.060	-0.1 ± 0.0
JointDW, GPT-6.1 Sol, data	GCG's detector	19	$0.95 \pm 0.03^*$	$0.947 \pm 0.028^*$	$-5.9 \pm 1.9^*$
JointDW, GPT-6.1 Sol, data	GPT-6.1 Sol, original presentation	19	-0.05 ± 0.03	-0.053 ± 0.028	0.0 ± 0.0
JointDW, Claude Sonnet 5.5, data	Lookahead, HiGHS dual	19	-0.11 ± 0.14	-0.088 ± 0.139	-92.7 ± 92.3
JointDW, Claude Sonnet 5.5, data	Lookahead, five duals	19	$-0.37 \pm 0.08^*$	$-0.351 \pm 0.082^*$	$2.1 \pm 0.7^*$
JointDW, Claude Sonnet 5.5, data	GCG's detector	19	$0.58 \pm 0.06^*$	$0.596 \pm 0.062^*$	$-3.7 \pm 1.9^*$
JointDW, Claude Sonnet 5.5, data	Claude Sonnet 5.5, original presentation	19	-0.11 ± 0.07	-0.088 ± 0.063	0.8 ± 0.4
All instances					
JointDW, GPT-6.1 Sol, data	Lookahead, HiGHS dual	59	0.10 ± 0.06	$0.105 \pm 0.051^*$	39.0 ± 57.1
JointDW, GPT-6.1 Sol, data	Lookahead, five duals	59	0.02 ± 0.06	0.016 ± 0.038	69.7 ± 53.3
JointDW, GPT-6.1 Sol, data	GCG's detector	59	$0.44 \pm 0.06^*$	$0.537 \pm 0.055^*$	$162.6 \pm 61.6^*$
JointDW, GPT-6.1 Sol, data	GPT-6.1 Sol, original presentation	59	$-0.11 \pm 0.04^*$	$-0.058 \pm 0.021^*$	$132.3 \pm 51.5^*$
JointDW, Claude Sonnet 5.5, data	Lookahead, HiGHS dual	59	$-0.11 \pm 0.06^*$	-0.077 ± 0.051	57.9 ± 52.8
JointDW, Claude Sonnet 5.5, data	Lookahead, five duals	59	$-0.20 \pm 0.05^*$	$-0.166 \pm 0.039^*$	$88.6 \pm 48.6^*$
JointDW, Claude Sonnet 5.5, data	GCG's detector	59	$0.23 \pm 0.05^*$	$0.355 \pm 0.048^*$	$181.5 \pm 63.0^*$
JointDW, Claude Sonnet 5.5, data	Claude Sonnet 5.5, original presentation	59	$-0.25 \pm 0.05^*$	$-0.145 \pm 0.036^*$	$151.3 \pm 52.2^*$

Table 25. Paired differences in solve time in seconds on MIPLIB-NL: JointDW or its interleaving with the lookahead order minus the covering or the lookahead order, mean ± standard error. Negative values favor JointDW or the interleaved order, and an asterisk marks a difference whose 95 percent bootstrap interval excludes zero, with each cell tested separately.

	JointDW minus		Interleaved minus	
Configuration	covering	lookahead	covering	lookahead
MIPLIB-NL, seconds, $s = 2$				
Claude Sonnet 5.5	$-21.61 \pm 9.80^{*}$	15.30 ± 20.60	$-34.53 \pm 11.92^{*}$	2.38 ± 13.83
GPT-6.1 Sol	$-42.97 \pm 12.34^{*}$	-6.06 ± 22.72	$-60.23 \pm 14.57^{*}$	-23.32 ± 17.74
Grok 4.20 (Reasoning)	$-24.46 \pm 11.87^{*}$	12.45 ± 21.09	$-42.03 \pm 14.52^{*}$	-5.11 ± 15.64
Grok 4.20 (Non-Reasoning)	-21.72 ± 12.92	15.19 ± 25.06	$-43.72 \pm 11.73^{*}$	-6.81 ± 16.34
DeepSeek-V4-Pro (Non-Thinking)	-8.22 ± 9.15	28.69 ± 21.01	$-24.80 \pm 13.36^{*}$	12.11 ± 14.82
MIPLIB-NL, seconds, $s = 3$				
Claude Sonnet 5.5	$-22.57 \pm 10.24^{*}$	24.19 ± 28.13	$-49.00 \pm 21.00^{*}$	1.68 ± 12.93
GPT-6.1 Sol	$-45.25 \pm 13.18^{*}$	3.03 ± 29.06	$-75.49 \pm 23.50^{*}$	-25.58 ± 17.37
Grok 4.20 (Reasoning)	$-33.08 \pm 13.37^{*}$	15.19 ± 29.37	$-62.92 \pm 22.37^{*}$	-11.40 ± 15.89
Grok 4.20 (Non-Reasoning)	-24.82 ± 15.54	23.45 ± 36.83	$-68.11 \pm 22.10^{*}$	-11.61 ± 16.81
DeepSeek-V4-Pro (Non-Thinking)	-7.61 ± 9.21	40.66 ± 30.18	$-38.75 \pm 22.93^{*}$	11.34 ± 14.53
MIPLIB-NL, seconds, $s = k - 1$				
Claude Sonnet 5.5	-13.42 ± 11.28	-9.41 ± 38.86	$-35.00 \pm 13.00^{*}$	-10.80 ± 21.11
GPT-6.1 Sol	$-31.40 \pm 12.23^{*}$	-26.89 ± 39.79	$-54.52 \pm 15.12^{*}$	-45.66 ± 33.95
Grok 4.20 (Reasoning)	-24.85 ± 13.92	-20.69 ± 41.41	$-49.17 \pm 13.07^{*}$	-23.06 ± 22.38
Grok 4.20 (Non-Reasoning)	$-28.96 \pm 12.28^{*}$	-24.60 ± 39.93	$-53.63 \pm 14.67^{*}$	-5.68 ± 16.12
DeepSeek-V4-Pro (Non-Thinking)	-3.50 ± 10.12	-4.39 ± 45.04	$-25.74 \pm 11.71^{*}$	16.16 ± 14.55

Table 26. Paired differences in the number of evaluations: JointDW or its interleaving with the lookahead order minus the covering or the lookahead order, mean ± standard error. Negative values favor JointDW or the interleaved order, and an asterisk marks a difference whose 95 percent bootstrap interval excludes zero, with each cell tested separately.

	JointDW minus		Interleaved minus	
Configuration	covering	lookahead	covering	lookahead
Templates, evaluations, $s = 2$				
Claude Sonnet 5.5	$-0.38 \pm 0.18^*$	0.47 ± 1.06	-0.90 ± 0.83	-0.04 ± 0.31
GPT-6.1 Sol	$-0.79 \pm 0.09^*$	0.07 ± 1.19	$-1.65 \pm 0.51^*$	-0.79 ± 0.82
Grok 4.20 (Reasoning)	$-0.38 \pm 0.15^*$	0.48 ± 1.10	-0.98 ± 0.77	-0.12 ± 0.38
Grok 4.20 (Non-Reasoning)	0.06 ± 0.19	0.92 ± 1.06	-0.45 ± 0.94	0.41 ± 0.23
DeepSeek-V4-Pro (Non-Thinking)	$0.28 \pm 0.10^*$	1.14 ± 1.05	-0.31 ± 0.93	$0.55 \pm 0.23^*$
Templates, evaluations, $s = 3$				
Claude Sonnet 5.5	-0.22 ± 0.21	0.19 ± 0.80	-0.40 ± 0.56	0.01 ± 0.29
GPT-6.1 Sol	$-0.60 \pm 0.18^*$	-0.20 ± 0.98	$-1.14 \pm 0.22^*$	-0.74 ± 0.76
Grok 4.20 (Reasoning)	-0.22 ± 0.18	0.18 ± 0.85	-0.50 ± 0.50	-0.09 ± 0.36
Grok 4.20 (Non-Reasoning)	0.36 ± 0.23	0.76 ± 0.88	0.03 ± 0.67	$0.44 \pm 0.22^*$
DeepSeek-V4-Pro (Non-Thinking)	$0.36 \pm 0.07^*$	0.77 ± 0.80	0.13 ± 0.65	$0.53 \pm 0.21^*$
Templates, evaluations, $s = k - 1$				
Claude Sonnet 5.5	-0.11 ± 0.23	-0.21 ± 0.63	0.10 ± 0.43	0.01 ± 0.29
GPT-6.1 Sol	$-0.66 \pm 0.15^*$	-0.76 ± 0.76	$-0.72 \pm 0.11^*$	-0.82 ± 0.74
Grok 4.20 (Reasoning)	-0.23 ± 0.19	-0.33 ± 0.63	-0.02 ± 0.37	-0.12 ± 0.35
Grok 4.20 (Non-Reasoning)	0.33 ± 0.22	0.24 ± 0.66	0.53 ± 0.52	$0.44 \pm 0.22^*$
DeepSeek-V4-Pro (Non-Thinking)	$0.32 \pm 0.07^*$	0.22 ± 0.64	0.55 ± 0.49	$0.46 \pm 0.20^*$
MIPLIB-NL, evaluations, $s = 2$				
Claude Sonnet 5.5	$-0.88 \pm 0.38^*$	0.03 ± 0.69	$-1.09 \pm 0.34^*$	-0.19 ± 0.43
GPT-6.1 Sol	$-1.29 \pm 0.49^*$	-0.39 ± 0.89	$-1.74 \pm 0.43^*$	-0.83 ± 0.81
Grok 4.20 (Reasoning)	$-0.66 \pm 0.33^*$	0.25 ± 0.60	$-0.93 \pm 0.45^*$	-0.03 ± 0.34
Grok 4.20 (Non-Reasoning)	-0.32 ± 0.34	0.59 ± 0.82	$-1.02 \pm 0.30^*$	-0.11 ± 0.47
DeepSeek-V4-Pro (Non-Thinking)	$-0.57 \pm 0.24^*$	0.33 ± 0.69	$-0.93 \pm 0.33^*$	-0.02 ± 0.39
MIPLIB-NL, evaluations, $s = 3$				
Claude Sonnet 5.5	$-0.50 \pm 0.21^*$	0.14 ± 0.72	$-0.84 \pm 0.30^*$	-0.14 ± 0.42
GPT-6.1 Sol	$-0.85 \pm 0.40^*$	-0.20 ± 1.00	$-1.55 \pm 0.37^*$	-0.79 ± 0.81
Grok 4.20 (Reasoning)	$-0.52 \pm 0.21^*$	0.13 ± 0.77	$-0.88 \pm 0.32^*$	-0.16 ± 0.43
Grok 4.20 (Non-Reasoning)	-0.09 ± 0.35	0.56 ± 0.95	$-0.95 \pm 0.27^*$	-0.22 ± 0.56
DeepSeek-V4-Pro (Non-Thinking)	-0.32 ± 0.20	0.33 ± 0.78	$-0.70 \pm 0.31^*$	0.01 ± 0.40
MIPLIB-NL, evaluations, $s = k - 1$				
Claude Sonnet 5.5	-0.08 ± 0.18	-0.38 ± 1.03	$-0.54 \pm 0.11^*$	-0.37 ± 0.61
GPT-6.1 Sol	-0.28 ± 0.24	-0.59 ± 1.09	$-0.81 \pm 0.07^*$	-0.97 ± 0.97
Grok 4.20 (Reasoning)	-0.15 ± 0.23	-0.47 ± 1.09	$-0.66 \pm 0.11^*$	-0.46 ± 0.64
Grok 4.20 (Non-Reasoning)	-0.01 ± 0.21	-0.34 ± 1.10	$-0.52 \pm 0.15^*$	0.04 ± 0.38
DeepSeek-V4-Pro (Non-Thinking)	0.04 ± 0.15	-0.29 ± 1.08	$-0.43 \pm 0.11^*$	0.15 ± 0.31